

Robust Candidate MaxWeight Certificates for Heterogeneous Compute Scheduling: A Scheduleurm Case Study

(Authors' names are not included for peer review)

Abstract. Modern compute clusters serve jobs whose rates depend on co-location, host load, accelerator memory, and placement. This makes the natural scheduling action a full configuration rather than a scalar resource allocation. We model the problem as a configuration-action stochastic processing network and prove a robust candidate MaxWeight theorem. The theorem keeps the full action space in the capacity statement and charges four implementable losses against explicit slack: candidate approximation, conservative service estimation, bounded penalties, and approximate optimization. A positive residual margin gives a finite-set Foster drift certificate under bounded conditional second moments. We construct candidate families from finite feature cells, obtaining an explicit cardinality bound and an $L\rho$ support-loss certificate, and derive finite-horizon expected-backlog bounds in addition to recurrence. We provide a Lean formalization of the fixed-family, queue-indexed statewise, candidate-cover, diagonal-scaling, and variable-duration frame statements. We instantiate the model in Scheduleurm, an operational research-cluster scheduler. The implementation records measured lower-service rows, admissible candidate configurations, penalty terms, oracle gaps, and production-admission status. Across 192 online-arrival measured-cache replay traces, the robust candidate improves geometric-mean makespan by 1.618000 $\times$ and mean flow by 4.180000 $\times$ over legacy Scheduleurm caps. On the same measured service cache, it is not Pareto-dominated by the registered policy-semantics baselines used to represent throughput, delay, and interference-control schedulers. The evidence supports a measured finite-slice stability and performance claim, with external full-stack and future-workload generalization left outside the main claim.

Key words: stochastic processing networks; MaxWeight scheduling; heterogeneous clusters; graphics-processing-unit co-location; robust optimization; formal verification; queueing networks

Subject classifications: Queues: scheduling; stochastic models: processing networks; computing: resource allocation

Area of review: Stochastic Models; Computing

1. Introduction

Schedulers for modern research clusters operate in a regime that is poorly described by a fixed resource vector. A reinforcement learning (RL) job can retain near-solo progress when several copies share a graphics processing unit (GPU), whereas a dense GPU kernel can lose finite-batch delay performance when the same GPU is aggressively multiplexed. Central processing unit (CPU)-heavy evaluation jobs stress host cores and memory rather than accelerator compute, and short control-plane jobs expose queueing overhead and fragmentation. These cases are not exceptions to

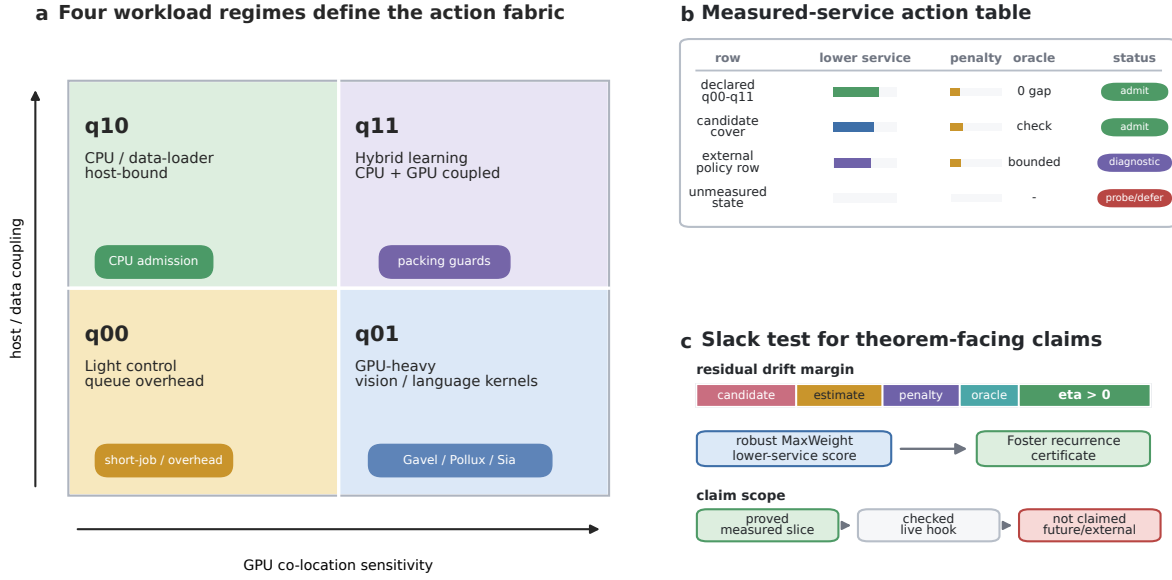
Scheduleurm certificate: four-regime actions to robust MaxWeight claims

Figure 1 Overview of the robust candidate MaxWeight certificate. A combinatorial full action space is represented by measured finite candidate actions across four workload quadrants. The scheduler scores candidates using the queue-weighted lower-service objective minus bounded penalties, and the stability certificate requires the residual slack to exceed candidate-cover loss, service-estimation error, penalty growth, and approximate-oracle error. External policy families are diagnostic finite actions on the same service cache, not universal full-stack baselines. The measured-service action table summarizes lower-service rows, bounded penalties, oracle gaps, and admission status.

a simple model; they are the model. The scheduler chooses global configurations whose service rates depend on the set of jobs, their placement, co-location profile, and local hardware state.

The operations-research difficulty is that the natural action is a global configuration. Enumerating all placements and co-location patterns is combinatorial, but reducing the problem to a small list of hand-picked “sweet spots” can invalidate stability claims. A policy may look efficient on a finite batch while silently using an infeasible profile, or it may be throughput-optimal in a model that is disconnected from the measured service map. Our starting point is therefore to keep the full action family in the mathematical statement and to treat the implemented candidate family as an approximation whose loss must be paid from explicit slack. Figure 1 gives the one-page certificate structure: candidate construction, lower-service scoring, slack accounting, and bounded claim boundaries are checked as separate objects.

The main theoretical contribution is a robust candidate MaxWeight theorem with complete slack accounting. The theorem states exactly how candidate approximation, conservative service

estimation, scheduling penalties, and approximate optimization consume capacity slack. When the remaining margin is positive and the stochastic primitives have bounded conditional second moments, the resulting policy satisfies finite-horizon expected-backlog bounds and a finite-set Foster recurrence certificate. A finite feature partition provides a constructive candidate family whose size is bounded by the number of active cells and whose support loss is at most $L\rho$. The stability result is stated for both fixed feasible families and state-dependent feasible families, the latter being the case relevant to live schedulers whose feasible actions depend on available jobs, alive nodes, and measured capacity boundaries. For migration and other event-driven actions, a separate variable-duration frame theorem preserves the same per-time slack accounting in cumulative service units and gives an embedded-chain recurrence certificate.

The empirical contribution is a theorem-condition evaluation, not a claim that a single implementation has solved cluster scheduling in full generality. We instantiate the theorem in *Scheduleurm*, an existing scheduler used for local research workloads, and report bounded pieces of evidence. First, measured service curves close declared finite slices for four workload buckets: local light-control tasks, GPU-heavy JAX numerical-computing matrix-multiplication tasks, local CPU-heavy tasks, and Robust-Ensemble Soft Actor-Critic (RE-SAC) / Bayesian Amnesic Piecewise-Robust (BAPR)-like hybrid RL tasks. Second, explicit replay compares robust candidate actions with legacy rules and diagnostic policy families on a common measured service cache, so all policies are evaluated against the same lower-service object. Third, holdout diagnostics, finite lower-service capacity certificates, ablation, production coverage, and live-state dry-run trace checks identify which assumptions are certified and which remain finite-slice claims. These results connect the proof obligations to scheduler artifacts; they do not replace a universal fabric-cover theorem or an exhaustive full-stack scheduler competition. Frozen FJSP, MMRCPS, and port holdouts additionally test whether the finite trajectory-action interface survives domain-specific feasibility and reconfiguration constraints; they are portability evidence, not automatic transfer of the server stochastic model.

The paper is organized as follows. We first state the claim scope and evidence contract, then position the work relative to scheduler and queueing literatures. We then introduce the configuration-action processing network, give the candidate approximation and robust MaxWeight stability theorem, describe the *Scheduleurm* instantiation, report the computational evidence, and close with the remaining calibration limits.

2. Scope and Contributions

Table 1 states the scope used throughout the paper. Each row connects one claim to the evidence used for it and to the stronger statement that the paper does not assert.

Table 1 Claim scope and evidence.

Claim	Evidence used here	Boundary
Robust candidate MaxWeight stability	Main-text proof, electronic-companion derivation, and Lean checks for exact and approximate oracle versions.	Holds only under the stated slack, lower-service, penalty, oracle, and moment conditions.
Variable-duration trajectory stability	Frame-level proof and Lean checks for cumulative service, bounded reconfiguration loss, duration bounds, and embedded recurrence.	A migration or port trace enters this theorem only after its cumulative stochastic service and moment model is certified.
Measured finite service domains	Declared q00/q01/q10/q11 service slices, mixed-portfolio replay, lower-service slack certificates, and probe/defer status for unknown states.	Does not imply positive service for unmeasured workloads, hardware states, or arbitrary future arrivals.
Scheduleum theorem objects	Measured service rows, candidate configurations, admission boundaries, penalty records, oracle-gap checks, and optional live-hook fields.	Does not make every production dispatch a theorem-grade global MaxWeight action.
Controlled and production evidence	Controlled launched/completed batches, strict completed-active production history, read-only shadow traces, and production readiness checks.	Supports admitted rows and controlled launches; raw telemetry, attempted-only rows, and future unknown jobs remain outside the theorem population.
External scheduler comparison	Registered policy-semantics baselines evaluated on the same measured service cache, plus scoped adapter or same-host runtime probes when available.	Does not claim direct full-stack superiority over every external scheduler or original multi-node deployment.
Cross-domain trajectory portability	Frozen FJSP integer ledgers, MMRCPSR renewal-stream and class-balance arrival-tape protocols, and separated port source-core and synthetic-migration certificates with exact registered-family oracle audits.	Does not claim arbitrary domain-solver dominance, industrial port validation, or automatic transfer of server stability.
Learning and hidden-regime extensions	Formal conditional results and finite active-bucket replay checks, forced exploration, switching, and detector-delay records.	Treated as extensions rather than main live performance or complete online-learning claims.

The table separates what is proved or measured from nearby claims that require additional experiments or assumptions.

3. Related Work

3.1. Deep-learning and heterogeneous cluster schedulers

Goodput-oriented deep-learning schedulers such as Gavel, Pollux, and Sia use measured or modeled throughput tables to allocate heterogeneous accelerators (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023). Fairness-oriented systems such as Themis study efficient sharing under fairness constraints (Mahajan et al. 2020). GPU sharing systems such as Gandiva, Salus, and IADeep expose or exploit fine-grained co-location and interference behavior (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Learning-based schedulers such as Decima and DL2 show another way to adapt scheduling decisions from traces or reinforcement learning (Mao et al. 2019, Peng et al. 2019). These systems motivate our measured-service replay baselines, but our theoretical question is different: we ask how a scheduler can restrict a combinatorial configuration action space to a candidate family while preserving a queueing-stability certificate with explicit loss terms.

Hybrid CPU/GPU placement and cluster allocation systems such as AlloX and related heterogeneous placement work also emphasize that accelerator scheduling cannot be reduced to GPU count alone (Le et al. 2020, Carvalho et al. 2024). Scheduleurm follows the same empirical lesson. Its q00, q10, and q11 buckets are declared local or node-bucket service certificates, not claims that all CPU-heavy or RL workloads have the same service curve.

3.2. Queueing-network control and learning

MaxWeight and backpressure policies are classical tools for stabilizing constrained queueing systems under capacity-region slack (Tassiulas and Ephremides 1992, Stolyar 2004, Neely 2010). Foster-Lyapunov and fluid-limit arguments also underlie positive recurrence and moment control in multiclass queueing networks (Dai and Meyn 1995, Meyn and Tweedie 2009). Our proof follows this lineage but adds three implementation-facing terms: candidate-set approximation, lower-service estimation, and approximate-oracle loss. Work on state-dependent processor sharing and workload-dependent admission control studies service rates that depend on the number or composition of active jobs (Gupta and Zhang 2022, Bekker and Borst 2006, Altman et al. 2006). The Scheduleurm co-location curves are a systems instance of the same phenomenon.

Learning-while-scheduling work, including MaxWeight with discounted upper confidence bound (UCB) and learning-based queueing controls, studies unknown service statistics under online exploration (Yang et al. 2023, Liu et al. 2022, Dai and Gluzman 2022). We do not claim a complete online learning theorem for the current Scheduleurm sampler. The formal artifact proves conditional high-probability lifting for active-bucket certificates. The current experiment artifact closes the deterministic finite active-bucket union bound and replay dwell/switching accounting, and adds a concrete deployable probability model: deterministic round-robin forced exploration over the finite active bucket set and a bounded two-window mean-shift detector. The certificate covers 64 active buckets and 120 replay scenarios with minimum forced samples 1561 per bucket, Hoeffding radius 0.0523, detector union tail bound 0.00564, and detection-delay bound 1024 decisions. A 2026-06-12 optional live-state probe further verifies that the theorem-facing live hook emits active-bucket sampler and regime-detector records while preserving the lower-service theorem oracle. This remains an opt-in extension path; the default live scheduler policy is not changed.

3.3. Finite configuration trajectories in operations research

Several classical operations-research scheduling domains share the same finite configuration-trajectory structure without sharing the same physical service model. In the flexible job-shop

scheduling problem (FJSP), a trajectory chooses an eligible machine and a position in the machine sequence for every operation; the Kacem instances provide heterogeneous processing times for these choices (Kacem et al. 2002). In the multi-mode resource-constrained project scheduling problem (MMRCPSP), each activity chooses a duration/resource mode subject to precedence and renewable and nonrenewable resource constraints; the Project Scheduling Problem Library (PSPLIB) supplies standard instances (Kolisch and Sprecher 1997). In port scheduling, berth positions, specific quay cranes, crane travel, and setup times form another configuration trajectory; the Berth Allocation and Specific Quay Crane Assignment Problem with travel and setup times (BACASP-S) formulation and instances make those costs explicit (Correcher et al. 2024). Section 7.4 uses these domains only to test portability of the finite-action selector. It does not identify their arrival, service, or reconfiguration laws with those of a GPU cluster, and it does not transfer the server recurrence theorem without a domain-specific stochastic model.

4. Model

4.1. Notation

Table 2 fixes the main slotted-model and replay notation. Frame-specific trajectory symbols are defined at first use in EC.6. Each mathematical symbol is reserved for one meaning; in particular, Q is the queue vector, q is an arbitrary nonnegative support weight, K_t is switching cost, and G_t is the risk or interference penalty.

4.2. Configuration actions

Let $\mathcal{I}$ be a finite set of job classes. Queue $Q_i(t) \in \mathbb{Z}_+$ counts the amount of unfinished work of class i at the beginning of slot t . A global configuration action $a \in \mathcal{A}^{\text{full}}$ specifies which jobs are served, which resource configuration each job receives, and where each job is placed. In a GPU cluster this includes, for example, the number of jobs sharing a GPU, CPU worker counts, memory pressure buckets, and host or device placement. The model intentionally treats these as one action rather than as independent per-job choices.

Given a regime z and exogenous randomness ω , action a generates a nonnegative potential-service vector $S(a, z, \omega) \in \mathbb{R}_+^{|\mathcal{I}|}$. Potential service is the work that the configuration could supply under backlog; it may be wasted when a queue is empty. We write $\mu^z(a) = \mathbb{E}[S(a, z, \omega)]$. For the main theorem we first suppress z and use $\mu(a)$; uniform-in-regime and average-regime extensions require additional switching assumptions and are not used as main empirical claims.

Table 2 Notation.

Symbol	Meaning
$\mathcal{I}$	finite set of job classes
i	job-class index
t	discrete decision slot
$Q_i(t)$	class- i backlog at slot t
$\mathbf{Q}(t)$	queue vector at slot t
$\mathcal{F}_t$	pre-decision information available to the scheduler at slot t
λ	arrival mean vector
$\mathbf{v}$	feasible service vector
$\mathbf{a}$	configuration action
$\mathbf{a}_t$	action selected at slot t
$\mathcal{A}^{\text{full}}$	full configuration-action family
$\mathcal{A}^{\text{cand}}$	fixed candidate-action family
$\mathcal{A}_t^{\text{cand}}$	statewise candidate-action family at slot t
$\mathcal{A}^{\text{meas}}$	measured finite action slice
Z_t	hidden service regime at slot t
$\chi(t)$	scheduler-visible state at slot t
ω_t	exogenous randomness at slot t
$A_i(t)$	class- i arrival at slot t
$S_i(t)$	class- i potential service offered at slot t
$D_i(t)$	class- i completed departure at slot t
$\mu^z(a)$	service mean of action a in regime z
$\mu(a)$	main-theorem service mean of action a
$\mu_t^L(a)$	conservative lower-service estimate for action a at slot t
$C(\mathcal{A})$	convex hull of service vectors generated by action family $\mathcal{A}$
$\Lambda(\mathcal{A})$	downward-closed load region of action family $\mathcal{A}$
$\text{Mix}(\mathcal{A})$	probability simplex over action family $\mathcal{A}$
x_a	stationary mixing mass assigned to action a
q	nonnegative support vector
$H_{\mathcal{A}}(q)$	support function of action family $\mathcal{A}$ in direction q
Φ	finite feature map for configuration actions
m	feature dimension
d_{Φ}	weighted fabric metric induced by Φ
w_r	weight of feature coordinate r
L	service Lipschitz envelope in the fabric metric
ρ	candidate-cover radius
π	projection from full actions to candidate actions
$K_t(a)$	switching cost of action a at slot t
$G_t(a)$	risk or interference penalty of action a at slot t
$P_t(a)$	total penalty $K_t(a) + G_t(a)$
P_0	additive penalty bound
β	queue-scaled penalty coefficient
θ_{prof}	dimensionless profile-penalty fraction in replay
δ	full-region slack
ϵ_{cand}	candidate-set support loss
ϵ_{est}	lower-service estimation loss
ϵ_{LCB}	normalized coordinate confidence radius
α_0	additive approximate-oracle error
α_1	queue-scaled approximate-oracle error
η	residual drift margin
B	second-moment bound
N	finite-set threshold
γ	negative-drift margin outside the finite set
V	quadratic Lyapunov function
ΔV	one-step Lyapunov drift
b	replay backlog count
k	measured co-location profile
$\mathcal{K}_i$	measured support envelope of profiles for class i
$k_i^*(b)$	replay profile selected for class i at backlog count b
$\mu_i(k)$	measured aggregate service of class i at profile k
$\mu_i^{\max}$	best measured aggregate service for class i in the same envelope
$r_i(a)$	coordinate confidence radius for service of action a
s_i	positive service-unit scale for class i
C_s	uniform upper bound on the service-unit scales s_i
c_i	task-native work units in one canonical replay job of class i

The notation avoids reusing one symbol for multiple meanings. Acronyms are expanded at first textual use and then used consistently.

Let $\mathcal{F}_t$ be the pre-decision filtration generated by the history through slot t , including $Q(t)$ and scheduler-visible state $\chi(t)$. The feasible families, service tables, penalties, and selected action a_t are $\mathcal{F}_t$ -measurable; arrivals and potential service are realized after the action is selected. Completed departure is $D_i(t) = \min\{Q_i(t), S_i(t)\}$, and the queue dynamics are equivalently

$$\begin{aligned} Q_i(t+1) &= Q_i(t) - D_i(t) + A_i(t) \\ &= [Q_i(t) - S_i(t)]^+ + A_i(t), \quad i \in \mathcal{I}. \end{aligned} \quad (1)$$

Here $A_i(t)$ is the class- i arrival in slot t . In the finite-support specialization used by the formal proof, the conditional distribution of $(A(t), S(t))$ depends on $Q(t)$ through a finite sample space. In the more general theorem it is enough to impose the conditional mean and second-moment bounds below with respect to $\mathcal{F}_t$.

4.3. Capacity and support functions

For a finite action family $\mathcal{A}$, define the service region

$$C(\mathcal{A}) = \left\{ \sum_{a \in \mathcal{A}} x_a \mu(a) : x \in \text{Mix}(\mathcal{A}) \right\}, \quad (2)$$

and its downward closure

$$\Lambda(\mathcal{A}) = \left\{ \lambda \in \mathbb{R}_+^{|\mathcal{I}|} : \exists v \in C(\mathcal{A}) \text{ with } \lambda \leq v \right\}. \quad (3)$$

The support function in the nonnegative orthant is

$$H_{\mathcal{A}}(q) = \max_{a \in \mathcal{A}} q^\top \mu(a), \quad q \in \mathbb{R}_+^{|\mathcal{I}|}. \quad (4)$$

Here $\text{Mix}(\mathcal{A})$ is the probability simplex over the action family $\mathcal{A}$, and x_a is the stationary mixing mass assigned to action a . The vector q is a nonnegative support weight; it is not the queue state unless explicitly specialized to $Q(t)$. If λ has coordinate slack $\delta > 0$ in the full region, then there is a stationary mixture $x \in \text{Mix}(\mathcal{A}^{\text{full}})$ such that $\lambda_i + \delta \leq \sum_a x_a \mu_i(a)$ for every i . Consequently,

$$q^\top \lambda + \delta \|q\|_1 \leq H_{\mathcal{A}^{\text{full}}}(q), \quad q \geq 0. \quad (5)$$

Inequality (5) is the bridge from capacity to MaxWeight drift; it is not an operational stability “if and only if” by itself.

4.4. Candidate families and fabric metrics

The implemented scheduler optimizes over $\mathcal{A}^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$. To avoid turning this computational restriction into an unaccounted model change, define a finite feature map $\Phi : \mathcal{A}^{\text{full}} \rightarrow \mathbb{R}^m$ and weighted ℓ_1 fabric metric

$$d_{\Phi}(a, a') = \sum_{r=1}^m w_r |\Phi_r(a) - \Phi_r(a')|. \quad (6)$$

The features may include co-location profile, node bucket, CPU bucket, memory pressure bucket, placement locality, and other scheduler-visible fabric attributes. We say that $\mathcal{A}^{\text{cand}}$ is a ρ -cover of $\mathcal{A}^{\text{full}}$ if for each full action a there is a candidate action $\pi(a) \in \mathcal{A}^{\text{cand}}$ with $d_{\Phi}(a, \pi(a)) \leq \rho$. If $\|\mu(a) - \mu(a')\|_{\infty} \leq L d_{\Phi}(a, a')$, then

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \|q\|_1, \quad q \geq 0. \quad (7)$$

The support-loss bound (7) is used as a theorem assumption unless a perturbation-profiling table certifies Φ, L, ρ . For exact measured finite slices in which the candidate family enumerates the measured actions, we set $\rho = 0$.

The next result makes the candidate construction explicit rather than taking a covering family as a primitive. Let $\mathcal{J}_{\Phi}$ be a finite collection of active feature cells and let $\kappa : \mathcal{A}^{\text{full}} \rightarrow \mathcal{J}_{\Phi}$ assign each full action to a cell. For every active cell, choose one full-action representative $\text{rep}(j)$, and define

$$\mathcal{A}^{\text{cand}} = \{\text{rep}(j) : j \in \mathcal{J}_{\Phi}\}. \quad (8)$$

PROPOSITION 1 (Finite-cell candidate construction). *Suppose every $a \in \mathcal{A}^{\text{full}}$ and its representative satisfy*

$$d_{\Phi}(a, \text{rep}(\kappa(a))) \leq \rho, \quad (9)$$

and the service map is L -Lipschitz in d_{Φ} . Then the family in (8) satisfies

$$|\mathcal{A}^{\text{cand}}| \leq |\mathcal{J}_{\Phi}| \quad (10)$$

and, for every $q \geq 0$,

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \|q\|_1. \quad (11)$$

Proof. The candidate family is the image of the active-cell set under rep , so its cardinality cannot exceed $|\mathcal{J}_{\Phi}|$. For each $a \in \mathcal{A}^{\text{full}}$, let $a' = \text{rep}(\kappa(a))$. By (9) and Lipschitzness, $\|\mu(a) - \mu(a')\|_{\infty} \leq L\rho$. Nonnegativity of q gives

$$q^{\top} \mu(a) \leq q^{\top} \mu(a') + L\rho \|q\|_1. \quad (12)$$

Taking the maximum over $a \in \mathcal{A}^{\text{full}}$ proves (11).

Proposition 1 separates two obligations. The feature partition and representative map are constructive algorithmic objects; the radius and service sensitivity remain calibration objects. Exact measured slices are the special case in which every active cell is represented exactly and $\rho = 0$. A nonzero empirical $L\rho$ claim additionally requires the perturbation profiling described in Section 7.

5. Robust Candidate MaxWeight

5.1. Policy

Let $\mu_t^L(a)$ be a conservative lower-service estimate for candidate action a at time t . Let $K_t(a) = K(a_{t-1}, a)$ model switching or rollback cost from the previous action to a , and let $G_t(a)$ model risk or interference penalties. The exact robust candidate MaxWeight rule is

$$a_t \in \arg \max_{a \in \mathcal{A}_t^{\text{cand}}} \{Q(t)^\top \mu_t^L(a) - K_t(a) - G_t(a)\}. \quad (13)$$

For practical schedulers, the selected action may be only approximately optimal. We assume the queue-scaled oracle condition relative to (13),

$$\begin{aligned} & \max_{a \in \mathcal{A}_t^{\text{cand}}} \{Q^\top \mu_t^L(a) - K_t(a) - G_t(a)\} \\ & \quad - \{Q^\top \mu_t^L(a_t) - K_t(a_t) - G_t(a_t)\} \\ & \leq \alpha_0 + \alpha_1 \|Q\|_1. \end{aligned} \quad (14)$$

The oracle loss in (14) is paid from the same slack budget as candidate loss and lower-service error. Exact maximization is the special case $\alpha_0 = \alpha_1 = 0$.

5.2. Main stability certificate

5.2.1. Assumptions We state the assumptions explicitly because each one corresponds to a measurable artifact in Scheduleum. All constants below are nonnegative and are fixed over the horizon under consideration.

Table 3 is the proof spine used below. The numbered lemmas give the main-text derivation; Appendix “Conventional Proof Details” then expands the same chain in electronic-companion style so that the reader can check the theorem without opening the Lean files.

ASSUMPTION 1 (Load slack). *The arrival mean vector λ has full-action support slack $\delta > 0$:*

$$q^\top \lambda + \delta \|q\|_1 \leq H_{\mathcal{A}^{\text{full}}}(q), \quad q \in \mathbb{R}_+^{|\mathcal{I}|}. \quad (15)$$

Table 3 Proof roadmap for Theorem 1.

Stage	Mathematical role	Term consumed
Capacity support	Convert full-action slack into a support-function margin.	None.
Candidate cover	Compare the full action family with the candidate family.	ϵ_{cand} , equal to $L\rho$ when a fabric metric is used.
Lower service	Replace unknown mean service by conservative service rows.	ϵ_{est} .
Penalty and oracle	Account for switching, risk, and approximate optimization.	$P_0, \beta, \alpha_0, \alpha_1$.
Drift margin	Combine the preceding losses into the residual slope η .	Requires $\eta > 0$.
Foster step	Use the quadratic Lyapunov inequality and second-moment bound to obtain negative drift outside a finite set.	B, N, γ and the local-return condition.

The table shows why each assumption is needed. Lean checks the algebraic implications; service, cover, oracle, and moment conditions are supplied by the measured Scheduleum certificates.

This is the support-function form of belonging to the interior of the full-action capacity base. It is a sufficient stability hypothesis; the operational necessity direction requires a separate conservation law.

ASSUMPTION 2 (Candidate cover and lower-service error). For every scheduler-visible state t and every $q \in \mathbb{R}_+^{|I|}$,

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1, \quad (16)$$

and

$$H_{\mathcal{A}_t^{\text{cand}}}(q) \leq \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) + \epsilon_{\text{est}} \|q\|_1. \quad (17)$$

When the fabric metric in Section 4.4 is certified, $\epsilon_{\text{cand}} = L\rho$. For an exact measured finite slice, $\epsilon_{\text{cand}} = 0$.

PROPOSITION 2 (Diagonal-scaled lower-confidence support loss). Let $\widehat{\mu}_i(a)$ estimate $\mu_i(a)$, let $r_i(a)$ be a coordinate confidence radius, and let $s_i \geq 0$ be a fixed service-unit scale. On the simultaneous confidence event

$$|\widehat{\mu}_i(a) - \mu_i(a)| \leq r_i(a), \quad 0 \leq r_i(a) \leq \epsilon_{\text{LCB}} s_i, \quad (18)$$

for every admitted $a \in \mathcal{A}_t^{\text{cand}}$ and class i , define

$$\mu_{t,i}^{\text{L}}(a) = \widehat{\mu}_i(a) - r_i(a). \quad (19)$$

Then $\mu_{t,i}^{\text{L}}(a) \leq \mu_i(a)$ and, for every $q \geq 0$,

$$H_{\mathcal{A}_t^{\text{cand}}}(q) \leq \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) + 2\epsilon_{\text{LCB}} \sum_i s_i q_i. \quad (20)$$

If $s_i \leq C_s$ for every i , Assumption 2 holds with

$$\epsilon_{\text{est}} = 2\epsilon_{\text{LCB}}C_s. \quad (21)$$

Proof. The upper side of (18) gives $\widehat{\mu}_i(a) - r_i(a) \leq \mu_i(a)$. Its lower side and the radius bound give

$$\mu_i(a) \leq \widehat{\mu}_i(a) + r_i(a) \leq \mu_{i,i}^L(a) + 2\epsilon_{\text{LCB}}s_i. \quad (22)$$

Multiplying by $q_i \geq 0$, summing over i , and maximizing over admitted actions proves (20). Finally, $\sum_i s_i q_i \leq C_s \sum_i q_i = C_s \|q\|_1$, which yields (21).

ASSUMPTION 3 (Penalty growth and approximate oracle). Let $P_t(a) = K_t(a) + G_t(a)$. For all $a \in \mathcal{A}_t^{\text{cand}}$,

$$0 \leq P_t(a) \leq P_0 + \beta \|Q(t)\|_1. \quad (23)$$

The selected action $a_t \in \mathcal{A}_t^{\text{cand}}$ satisfies the approximate-oracle condition

$$\begin{aligned} \max_{a \in \mathcal{A}_t^{\text{cand}}} \{Q(t)^\top \mu_t^L(a) - P_t(a)\} \\ - \{Q(t)^\top \mu_t^L(a_t) - P_t(a_t)\} \leq \alpha_0 + \alpha_1 \|Q(t)\|_1. \end{aligned} \quad (24)$$

Exact robust MaxWeight is the special case $\alpha_0 = \alpha_1 = 0$.

ASSUMPTION 4 (Stochastic domination and bounded second moments). The action a_t , state-wise action families, lower-service map, and penalties are $\mathcal{F}_t$ -measurable. For every class i ,

$$\mathbb{E}[A_i(t) \mid \mathcal{F}_t] \leq \lambda_i, \quad (25)$$

and

$$\mu_{i,i}^L(a_t) \leq \mathbb{E}[S_i(t) \mid \mathcal{F}_t] \quad (26)$$

almost surely. Moreover,

$$\mathbb{E} \left[\frac{1}{2} \sum_i \{A_i(t)^2 + S_i(t)^2\} \middle| \mathcal{F}_t \right] \leq B. \quad (27)$$

ASSUMPTION 5 (Countable-state recurrence condition). For a positive-recurrence conclusion, the augmented controlled state $X(t) = (Q(t), \chi(t))$ is a Markov chain on a countable state space. The set

$$C_N = \{x : \|Q(x)\|_1 \leq N\} \quad (28)$$

used by the drift certificate is finite and nonempty, and the corresponding closed class satisfies the usual local-return condition. Without these conditions we claim only the finite-set negative-drift certificate.

5.2.2. Deterministic slack accounting

LEMMA 1 (Support slack after candidate and estimation losses). *Under Assumptions 1–2, for every t and $q \in \mathbb{R}_+^{|I|}$,*

$$\begin{aligned} \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) &\geq q^\top \lambda \\ &+ (\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}}) \|q\|_1. \end{aligned} \quad (29)$$

Proof. By Assumption 1,

$$q^\top \lambda + \delta \|q\|_1 \leq H_{\mathcal{A}^{\text{full}}}(q). \quad (30)$$

Assumption 2 gives

$$\begin{aligned} H_{\mathcal{A}^{\text{full}}}(q) &\leq H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 \\ &\leq \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) \\ &+ (\epsilon_{\text{cand}} + \epsilon_{\text{est}}) \|q\|_1. \end{aligned} \quad (31)$$

Rearranging proves the claim.

LEMMA 2 (Selected lower-service margin). *Under Assumptions 1–3, for $Q = Q(t)$,*

$$\begin{aligned} Q^\top \mu_t^{\text{L}}(a_t) &\geq Q^\top \lambda \\ &+ (\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}} - \beta - \alpha_1) \|Q\|_1 \\ &- (P_0 + \alpha_0). \end{aligned} \quad (32)$$

Proof. Let $M(Q) = \max_{a \in \mathcal{A}_t^{\text{cand}}} Q^\top \mu_t^{\text{L}}(a)$. Since $P_t(a) \leq P_0 + \beta \|Q\|_1$ for every candidate,

$$\begin{aligned} \max_{a \in \mathcal{A}_t^{\text{cand}}} \{Q^\top \mu_t^{\text{L}}(a) - P_t(a)\} \\ \geq M(Q) - P_0 - \beta \|Q\|_1. \end{aligned} \quad (33)$$

The approximate-oracle condition therefore implies

$$\begin{aligned} Q^\top \mu_t^{\text{L}}(a_t) - P_t(a_t) &\geq M(Q) - P_0 - \beta \|Q\|_1 \\ &- \alpha_0 - \alpha_1 \|Q\|_1. \end{aligned} \quad (34)$$

Because $P_t(a_t) \geq 0$, the left-hand side is at most $Q^\top \mu_t^{\text{L}}(a_t)$. Applying Lemma 1 with $q = Q$ gives the stated inequality.

LEMMA 3 (Quadratic one-step bound). *For $V(Q) = \frac{1}{2} \sum_i Q_i^2$ and nonnegative vectors Q, A, S ,*

$$\begin{aligned} V([Q - S]^+ + A) - V(Q) \\ \leq \frac{1}{2} \sum_i (A_i^2 + S_i^2) + Q^\top A - Q^\top S. \end{aligned} \quad (35)$$

Proof. It is enough to prove the coordinate inequality. For $q, a, s \geq 0$, $([q - s]^+)^2 \leq (q - s)^2$ and $[q - s]^+ \leq q$. Hence

$$\begin{aligned} ([q - s]^+ + a)^2 &\leq ([q - s]^+)^2 + a^2 + 2a[q - s]^+ \\ &\leq q^2 + s^2 + a^2 + 2q(a - s). \end{aligned} \quad (36)$$

Subtracting q^2 , dividing by two, and summing over coordinates gives the result.

THEOREM 1 (Robust candidate MaxWeight stability certificate). *Consider a finite system with job classes $\mathcal{I}$ and queue dynamics (1). Suppose Assumptions 1–4 hold. If*

$$\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1) > 0, \quad (37)$$

then for $V(Q) = \frac{1}{2} \sum_i Q_i^2$ and $\Delta V(t) = V(Q(t+1)) - V(Q(t))$,

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] &\leq B + P_0 + \alpha_0 \\ &\quad - \eta \|Q(t)\|_1. \end{aligned} \quad (38)$$

Consequently, for any N satisfying

$$B + P_0 + \alpha_0 + \gamma \leq \eta N \quad (39)$$

for some $\gamma > 0$, the process has a negative expected drift outside C_N . If Assumption 5 also holds, the standard countable-state Foster criterion gives positive recurrence of the corresponding closed communicating class.

Proof. Apply Lemma 3 with $(Q, A, S) = (Q(t), A(t), S(t))$ and condition on $\mathcal{F}_t$. Using Assumption 4,

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] &\leq B + Q(t)^\top \mathbb{E}[A(t) \mid \mathcal{F}_t] \\ &\quad - Q(t)^\top \mathbb{E}[S(t) \mid \mathcal{F}_t] \\ &\leq B + Q(t)^\top \lambda - Q(t)^\top \mu_t^L(a_t). \end{aligned} \quad (40)$$

Write $D_t = Q(t)^\top \lambda - Q(t)^\top \mu_t^L(a_t)$. Lemma 2 gives

$$\begin{aligned} D_t &\leq P_0 + \alpha_0 \\ &\quad - (\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}} - \beta - \alpha_1) \|Q(t)\|_1. \end{aligned} \quad (41)$$

Combining the two displays proves

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] &\leq B + P_0 + \alpha_0 \\ &\quad - \eta \|Q(t)\|_1. \end{aligned} \quad (42)$$

If $B + P_0 + \alpha_0 + \gamma \leq \eta N$, the drift is at most $-\gamma$ whenever $\|Q(t)\|_1 > N$. This is the finite-set Foster drift certificate. Under Assumption 5, the usual countable-state Foster theorem yields positive recurrence of the closed class.

COROLLARY 1 (Finite-horizon expected-backlog bound). *Under the hypotheses of Theorem 1, suppose the Lyapunov variables are integrable. For every integer $T \geq 1$,*

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|Q(t)\|_1] \leq \frac{B + P_0 + \alpha_0}{\eta} + \frac{\mathbb{E}[V(Q(0))]}{\eta T}. \quad (43)$$

Consequently,

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|Q(t)\|_1] \leq \frac{B + P_0 + \alpha_0}{\eta}. \quad (44)$$

Proof. Take expectations in the conditional drift bound of Theorem 1, sum from $t = 0$ to $T - 1$, and apply the tower property. The Lyapunov terms telescope and $V \geq 0$, so

$$\begin{aligned} \eta \sum_{t=0}^{T-1} \mathbb{E}[\|Q(t)\|_1] &\leq T(B + P_0 + \alpha_0) \\ &\quad + \mathbb{E}[V(Q(0))]. \end{aligned} \quad (45)$$

Dividing by ηT gives (43); taking $T \rightarrow \infty$ gives (44).

COROLLARY 2 (Statewise feasible families). *Let the full family, candidate family, true service, lower service, feature map, and penalty be indexed by the scheduler-visible state $X(t) = (Q(t), \chi(t))$:*

$$\begin{aligned} \mathcal{A}_t^{\text{full}} &= \mathcal{A}^{\text{full}}(X(t)), \quad \mathcal{A}_t^{\text{cand}} = \mathcal{A}^{\text{cand}}(X(t)), \\ \mu_t &= \mu(X(t), \cdot), \quad \mu_t^{\text{L}} = \mu^{\text{L}}(X(t), \cdot), \\ \Phi_t &= \Phi(X(t), \cdot), \quad P_t = P(X(t), \cdot). \end{aligned} \quad (46)$$

Here $\chi(t)$ records scheduler-visible availability, pinned jobs, node health, and measured capacity boundaries. Suppose the load-slack, calibrated candidate-cover, lower-service, penalty, oracle, and stochastic-moment conditions hold pointwise for every admitted state $X(t)$, with the same constants $\delta, \epsilon_{\text{cand}}, \epsilon_{\text{est}}, P_0, \beta, \alpha_0, \alpha_1, B$. Then the drift bound and the finite-horizon backlog bounds in Theorem 1 and Corollary 1 hold unchanged. If Assumption 5 holds for the augmented state, the same finite-set recurrence conclusion follows.

Proof. The proof of Lemmas 1–2 is pointwise in all six objects in (46). Conditioning on $\mathcal{F}_t$ fixes the realized full and candidate families, service maps, features, penalties, and action. The

same inequality chain therefore gives the same lower-service margin and quadratic drift bound. Uniformity of the constants over the admitted statewise family permits the tower-property and Foster steps without introducing an additional state-dependent loss.

The Lean 4 theorem prover artifact (de Moura and Ullrich 2021) proves the fixed-family theorem and the queue-indexed specialization of Corollary 2. In that specialization the full family, candidate family, true and lower-service maps, feature map, and penalty all depend on the integer queue snapshot. The general $\mathcal{F}_t$ -conditional form above is the conventional augmented-state extension; its recurrence conclusion additionally uses Assumption 5. The artifact also proves a bounded finite-support specialization in which B is constructed from coordinate sample bounds and a finite-horizon telescoping backlog inequality. The paper theorem numbers are not Lean identifiers. Appendix Table 16 gives the exact crosswalk for Theorem 1, Corollary 2, the coordinate-scaled lower confidence bound result, and the operational sandwich boundary. *Scope.* The theorem applies when the stated slack, lower-service, penalty, oracle-error, and moment assumptions are certified; it is not by itself a certificate for every scheduler dispatch.

5.3. Proof decomposition

The preceding lemmas isolate the mathematical obligations that must be certified by the scheduler. Proposition 1 constructs a bounded candidate family; Proposition 2 converts coordinate confidence radii into a support loss; Lemma 1 is the capacity and approximation layer; Lemma 2 is the robust oracle layer; Lemma 3 is the queueing layer; and Theorem 1 with Corollary 1 is their Foster-Lyapunov composition. The Lean formalization mirrors this separation. The operational necessity direction is kept separate: an “operationally stabilizes” predicate is load-certified by a model that explicitly encodes the mean arrival vector, and zero-slack necessity follows from a conservation-law argument rather than from the definition of the predicate.

6. Scheduleum Instantiation

6.1. Measured action slices

Scheduleum is an existing scheduler for local and remote research jobs. The new algorithmic layer deliberately separates the theorem-facing policy from the legacy launcher. The theorem-facing policy builds global or statewise candidate families with a queue vector, lower-service vector, penalty units, and robust MaxWeight score. Service profiling, replay, baseline comparison, slack accounting, and oracle-gap checks are implemented outside the legacy scheduler. The live scheduler

Table 4 Declared measured finite action slices.

Bucket	Dominant service bottleneck	Measured feasible family	Excluded boundary	Use in replay and certificate
q00 light control	control-plane overhead and short-job fragmentation	profiles 1–13	profile 14	replay profile 13; legacy cap 1
q01 GPU-bound compute	accelerator co-location and memory pressure	profiles 1–8	none observed in this node bucket	replay profile 1; profiles 4 and 8 retained for support and LCB certificates; legacy cap 3
q01 CNN ResNet-50	convolutional training throughput and tail drain	profiles 1–7	profile 8 placement/stability boundary	static and rolling replay profile 5; LCB support profile 7; legacy cap 3
q01 LLM DistilGPT-2	transformer forward-pass concurrency and memory pressure	profiles 1–6 and 8	profiles 10–11 boundary rows	replay profile 8; sparse online arrivals drain at profile 1; legacy cap 3
q10 CPU-host bound	host-core and memory-bandwidth contention	profiles 1–9	profile 10	replay profile 8; legacy cap 9
q11 CPU/GPU coupled RL	coupled host and accelerator progress	profiles 1–5	profile 6 thread/placement boundary on node007	replay profile 5; mixed-portfolio support includes profile 5; legacy cap 5

A profile is robust feasible only after measured lower-service and admission checks. The boundary column records the first excluded profile when a boundary was observed. Lower confidence bound (LCB) certificates may retain support profiles that are not selected by the finite-batch replay policy.

exposes opt-in hooks: a one-task placement scorer and a global-batch soft-hint path. The global-batch hook computes a bounded candidate configuration and returns only a preferred-node hint to the legacy launcher. Final placement therefore still passes through the existing resource and launch checks. This design lets us test the theorem-facing policy without making it the default production dispatcher.

Table 4 maps each measured workload bucket to the finite action slice used by the theorem-facing policy. A capacity boundary is a measured or live sanity profile that is not usable as a robust feasible action. Once profile k is a boundary, profile k and all higher profiles are excluded from the theorem-facing candidate family for that workload bucket. The table is therefore both an experimental summary and a definition of the declared finite slices used in the capacity and replay claims.

The replay policy uses a queue-adaptive robust MaxWeight penalty selector rather than a fixed co-location profile. For a workload with backlog count b , it first restricts profiles to a measured support envelope $\mathcal{K}_i$ and then chooses

$$k_i^*(b) \in \arg \max_{k \in \mathcal{K}_i} \{b \widehat{\mu}_i(k) - \theta_{\text{prof}} \widehat{\mu}_i^{\max} k\}, \quad (47)$$

$$\theta_{\text{prof}} = 0.10.$$

Here $\mathcal{K}_i$ is the measured support envelope for class i , $\widehat{\mu}_i(k)$ is the measured aggregate service for class i at profile k , and $\widehat{\mu}_i^{\max}$ is the best measured aggregate service in the same envelope. Thus

Table 5 Theorem-policy crosswalk in Scheduleurm.

Theorem object	Scheduleurm representation	Interpretation
Queue vector Q	backlog vector recorded for each replay or live slot	Defines the backlog weights in $Q^\top \mu^L(a)$.
Candidate action $a \in \mathcal{A}_t^{\text{cand}}$	finite placement, co-location, admission, and drain-rule configuration	Represents a global or statewise placement/co-location configuration for the theorem-facing oracle.
State index $\chi(t)$	alive nodes, pinned jobs, queued jobs, available-device state, and measured admission status	Selects the statewise candidate family without changing the fixed-family drift constants.
Lower service $\mu^L(a)$	conservative measured-service row	Conservative service vector used in the drift inequality.
Admission and probe state	measured bucket, capacity boundary, lower-confidence status, and probe/defer flag	Determines whether a row enters the theorem-facing population or receives zero theorem service.
Penalty $K_t(a) + G_t(a)$	bounded switching, risk, and delay-control terms	Consumes bounded or queue-scaled slack.
Oracle error (α_0, α_1)	oracle-gap and support-gap reports	Measures approximate maximization loss relative to the theorem objective.
Live placement hook	optional one-task and global-batch scheduler hint	Integration point; theorem use requires candidate, service, penalty, and oracle-gap fields.

The theorem-facing rows expose robust MaxWeight score semantics explicitly; the live hook is an integration surface and not, by itself, a stability certificate.

the replay candidate uses lower-interference profiles such as q01 profile 1 when they stay within the measured support slack, while support certificates retain actions such as q01 profiles 4 and 8 for capacity accounting. The optional statewise drain is lower-service guarded and no-upshift: it may only reduce a profile when the measured lower-service and waiting-backlog checks certify no loss relative to the base action. This implementation matches the approximate-oracle theorem: the support envelope preserves the drift action at large Q , and the finite-batch delay preference is a bounded profile penalty that is paid from the same slack budget as $K_t + G_t$.

Table 5 maps the theorem objects to the Scheduleurm implementation. Rows generated by the theorem-facing policy expose the robust MaxWeight objective directly. Rows generated by the live hook require the same candidate, service, penalty, and oracle-gap fields before they are used as theorem evidence.

6.2. Theorem evidence populations

Scheduleurm distinguishes three populations. The explicit task-list replay population is used for performance comparison. The completed-active production population is used for service-map and oracle-gap checks. Raw scheduler history is treated as telemetry because it contains cancelled, attempted-only, benchmark, and externally adopted jobs.

The completed-active Scheduleurm-controlled production population has 3,437 mapped records out of 3,437, positive mapped-slice capacity slack, and a service-map robust MaxWeight lower-service bridge with $\alpha_0 = \alpha_1 = 0$. A longer live-state dry-run covers 128 placement slots and 767

candidate rows without writing the queue or launching tasks. A separate bounded q01/q11 SchedulermBench run exercises the robust lower-service hook on real tasks. These checks show that the theorem-facing fields can be generated from the scheduler state. They do not treat raw telemetry as a clean arrival stream.

7. Computational Evidence

7.1. Explicit task-list replay

The main benchmark uses one explicit task list per quadrant and one mixed portfolio. All policies receive the same jobs, total work units, resource-pool counts, arrival times, and measured service cache. Static arrivals are used for the first comparison because all-job completion time is then unambiguous. The online-arrival experiment then runs nominal-load Poisson and bursty traces and a four-level load sweep at factors 0.50, 0.70, 0.85, and 0.95, with three random seeds for every hardware-local task set.

The replay is a joint event simulation, not a sum of independent single-workload simulations. All workload classes assigned to one hardware scenario compete for the same physical CPU or GPU lanes. Before robust MaxWeight scoring, each class's residual work and every service coordinate are divided by the same fixed positive canonical-work scale c_i . The resulting coordinates are unfinished canonical jobs and canonical jobs per unit time, so the queue recursion and the main theorem are applied in those coordinates. Changing the numerical task-native unit of one workload, together with its c_i , therefore cannot change an action ordering. This unit conversion is separate from the LCB error scales s_i in Proposition 2. A directional loaded action requires one resident job and one target job on the same lane. Its frame duration is the target's task-native natural-completion time, while the resident receives only its certified overlap lower service; any unfinished resident returns to the queue with its remaining work. Migration likewise occupies one lane for its measured variable-duration frame. These rules make resource competition, co-location direction, and frame duration explicit and connect the replay semantics to Proposition 3.

Table 6 compares the current candidate policy with the legacy Schedulerm caps on 24 closed-batch traces. The metric is completion of all jobs in the task list, not the time to finish a single job. In the task names, convolutional neural network (CNN) denotes the vision workload family and large language model (LLM) denotes the language-model workload family.

Scope. Table 6 compares all-job completion on declared task lists using measured service rows.

Table 7 summarizes 192 online-arrival comparisons spanning nominal Poisson, nominal bursty, and Poisson/bursty load-sweep traces. The adaptive candidate completed every job in every admitted

Table 6 Candidate policy versus legacy Scheduleum on explicit task lists.

Hardware-local task list	Traces	Makespan	Mean flow	p90 flow
q00 light control	1	11.814000	11.758000	11.814000
q01 GPU compute	1	1.470000	1.560000	1.555000
q01 CNN ResNet-50	1	1.259000	1.173000	1.164000
q01 LLM DistilGPT-2	1	2.723000	2.622000	2.669000
q01 GPU portfolio	1	1.613000	1.599000	1.624000
q10 CPU-host bound	1	1.539000	1.535000	1.534000
q11 CPU/GPU coupled	1	1.005000	1.000000	1.000000
Mixed portfolio	1	1.012000	1.162000	0.994000

Ratios are legacy divided by candidate, so larger is better for Scheduleum candidate. Each row is derived from the same hash-bound schema-v2 replay as the online, SOTA-style, ablation, migration, and slack summaries. p90 is the 90th percentile of job flow time.

Table 7 Online replay distribution over 192 comparisons.

Metric	Geomean/mean	Median	Min	5%	95%
Legacy / candidate makespan	1.618000	1.248000	1.003000	1.007000	9.835000
Legacy / candidate mean flow	4.180000	2.492000	1.000000	1.027000	179.178000
Candidate mean backlog jobs	7.783000	8.318000	1.099000	1.763000	27.657000
Candidate max backlog jobs	23.654000	24.000000	4.000000	7.000000	65.000000

No online scenario Pareto-dominated the candidate. No ablated policy Pareto-dominated the full candidate. Some single-metric losses against external-policy diagnostics exceed the 0.5% tolerance, but none is a Pareto-dominance violation.

Table 8 Online replay strata from the same hash-bound population.

Stratum	Comparisons	Makespan geomean	Mean-flow geomean	Makespan min	Mean-flow min
<i>Arrival-family strata</i>					
bursty	64	—	—	—	—
load sweep	64	—	—	—	—
Poisson	64	—	—	—	—
<i>Quadrant strata</i>					
q00	48	—	—	—	—
q01	48	—	—	—	—
q10	48	—	—	—	—
q11	48	—	—	—	—

Ratios are legacy divided by candidate. The load-sweep family contains both Poisson and bursty arrivals at the four registered load factors.

trace. Relative to legacy caps, its geometric-mean improvement was $1.618000\times$ for makespan and $4.180000\times$ for mean flow. The minimum scenario-level ratios were $1.003000\times$ and $1.000000\times$, respectively. The largest observed candidate queue backlog was 90.000000 jobs, and the largest time-weighted mean queue backlog was 48.112000 jobs. All ETA-dependent rows use task-native natural-completion models; history fallback is excluded from the admitted population.

The ablation experiment isolates support-only and delay-only scoring, bounded penalties, state-wise loaded actions, migration actions, and the profile tie-break rule. Table 9 evaluates 6 pre-

Table 9 Component ablations on the unified replay population.

Ablated policy	Comparisons	Makespan ratio	Mean-flow ratio	Dominance count
Delay-only score	24	—	—	0
High-profile tie-break	24	—	—	0
No bounded penalty	24	—	—	0
No loaded-action ledger	24	—	—	0
No migration actions	24	—	—	0
Support-only score	24	—	—	0

Ratios are ablated policy divided by the full policy, so values above one favor the full policy. Dominance count is the number of traces on which the ablation strictly Pareto-dominates the full policy.

registered removals on the same cache and traces as the full policy. The full policy is Pareto-dominated in 0 ablation comparisons; the table reports all single-metric changes rather than treating a non-dominance result as evidence that every module improves every metric.

7.2. Policy-semantics replay baselines

No single external scheduler covers all four Scheduleurm quadrants and the mixed portfolio without changing the workload model. For this reason the external-scheduler evidence is organized as a diagnostic layer rather than as the central claim of the paper. We compare against policy-semantics replay baselines on the same measured service cache: a throughput-table goodput policy representing Gavel, Pollux, and Sia-like schedulers (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023); a finite-batch delay oracle representing shortest remaining processing time (SRPT), Gittins, and shortest expected remaining processing time (SERPT)-style policies (Scully et al. 2020); and an interference guard representing IADeep/Salus/Gandiva-like co-location control (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Because every row is evaluated through the same measured lower-service map, these baselines test whether the finite candidate family is missing an obvious scheduling semantic; they are not direct binary executions of the external systems.

We also register 11 non-adjacent scheduler families as finite actions in the Scheduleurm+external action union. This is the form needed by the candidate-set theorem: once an external policy family is represented by a lower-service row and a bounded penalty, it can be evaluated by the same robust selector. Several systems also have scoped same-host runtime checks, and Decima has a same-domain Spark directed acyclic graph (DAG) simulator check. These checks support traceability. They are not used as the central comparison because the paper’s optimization claim is about measured configuration actions, not about deploying every external control plane under identical production conditions.

Table 10 Policy-semantics aggregate on the identical measured service cache.

Policy-semantics baseline	Comparisons	Makespan ratio	Mean-flow ratio	Dominance count
Gavel finish-time fairness	16	0.999996	1.000010	0
Gavel/Pollux/Sia table goodput	16	1.006400	1.000300	0
IADeep/Salus interference guard	16	1.002500	0.999960	0
Quadrant composite	16	1.006400	1.000300	0
Salus/IADeep packing guard	16	0.999996	1.000010	0
Sia/Pollux resource-adaptive	16	1.000002	1.000006	0
SRPT/Gittins delay oracle	16	1.006300	1.000200	0

Entries are baseline divided by candidate, so values above one favor the candidate. Dominance count is evaluated trace by trace under the registered 0.5% replay tolerance. These are policy-semantics rows, not full-stack external-system executions.

The multi-node check covers Scheduleurm's own measured history. Two GPU nodes were reachable during the read-only fabric probe. The resulting candidate set contained 18 lower-service rows and 626 disjoint robust-MaxWeight configurations, including a selected two-node configuration with $\alpha_0 = \alpha_1 = 0$. A separate scheduler-history study found 4,058 strict theorem-admitted launches, 4,047 completions, 35 workload domains, 13 nodes, three GPU nodes, and no strict unadmitted launched rows. This supports a Scheduleurm-native multi-node history claim. It does not require claiming that each external scheduler has been run through its original multi-node control plane.

Table 10 reports all 7 registered policy-semantics families over the hash-bound hardware-local replay population. The result is stated directly from the exported diagnostic: Scheduleurm is not Pareto-dominated by any registered SOTA-style policy on any replay trace. This does not imply that Scheduleurm wins each metric on each trace; the per-policy ratios and strict-dominance counts retain those distinctions.

Figure 2 visualizes the same policy-semantics comparison as a four-quadrant cost frontier. Each panel normalizes a registered SOTA-style policy by the fixed Scheduleurm candidate: the horizontal coordinate is mean-flow cost divided by Scheduleurm's mean-flow cost and the vertical coordinate is makespan cost divided by Scheduleurm's makespan cost. Thus the Scheduleurm policy is fixed at (1, 1), and points above or to the right of the star are costlier under the same measured service cache. The quadrant labels also show that the best SOTA-style family changes across workload regimes. The figure is intentionally scoped to measured-cache policy-semantics comparison and is not a direct external binary benchmark.

We also evaluate the stronger theory-facing construction in which external policy-family actions are not merely baselines but are included in the candidate set. The selector evaluates the Scheduleurm candidate action together with the throughput-table, delay-oracle, interference-guard, finish-time-fairness, resource-adaptive, packing-guard, and measured bridge-action families under the same

Different SOTA-style policy families win different quadrants; Scheduleurm is not Pareto dominated on the measured-cache replay

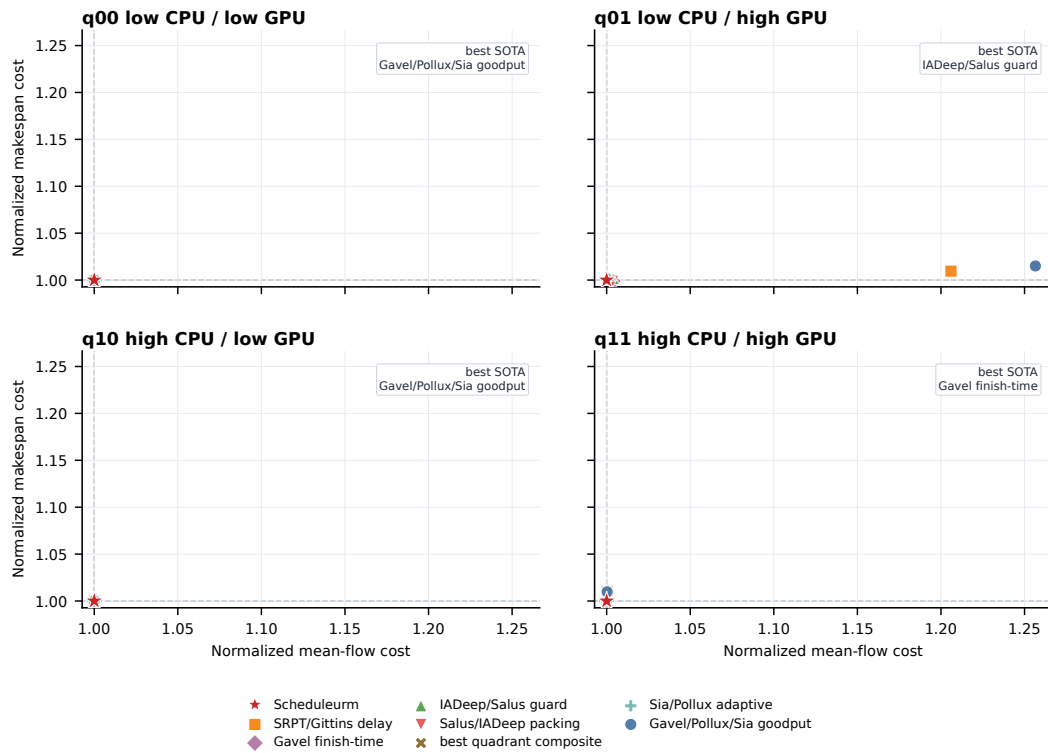


Figure 2 Four-quadrant measured-cache SOTA-style comparison. In each panel, the red star is Scheduleurm at (1, 1). Other markers are registered SOTA-style policy families evaluated on the same task-native measured service cache, with x = normalized mean-flow cost and y = normalized makespan cost, both expressed as policy cost divided by Scheduleurm cost. The box in each panel reports the scenario-level best SOTA-style family for that quadrant. The comparison is a policy-semantics replay on shared measured service rows, not direct full-stack external binary execution.

measured service units. The action is not deduplicated by co-location profile alone: a candidate is a finite configuration trajectory containing the placement/profile, resource partition, admission order, and statewise drain rule. This distinction matters on the ResNet-50 bucket, where the same profile supports different finish-time and interference-drain trajectories. The unified replay evaluates that finite Scheduleurm-plus-external action family directly instead of importing aggregate completion times from an older replay. For every registered trace, the selector enumerates only actions with a hardware-local lower-service row and applies the same canonical scaling, penalty accounting, lane feasibility, and variable-duration completion model. Table 10 therefore reports the action-union comparison on the same population as Tables 6–8; no separately normalized table of absolute times is mixed into the claim.

The strict measured-cache claim is separate from a direct-system claim. The final fixed candidate treats a configuration action as a measured trajectory, not merely as a co-location profile: the action

identifies the hardware class, resource state, resident/target direction, admission order, and drain rule. Across q00, q01, q10, and q11, the exported trace-level comparison records whether any registered baseline Pareto-dominates that fixed candidate. The statement remains a measured-cache policy-semantics claim, not a direct full-stack superiority claim over external binaries.

The implementation layer includes the same action-union rule as an optional candidate policy. It adds a Pareto-slack Scheduleurm+external selector, state-dependent marginal service rows, bounded global batch lookahead, reusable online lower-confidence-bound (LCB) estimated time remaining (ETA) estimates, and external-policy action families for finish-time fairness, adaptive goodput, and packing guards. These additions are useful because the measured service curves are not monotone in load. In the controlled slice, a small CUDA task with a high video random-access memory (VRAM) resident job reached $1.095\times$ the empty reference rate, and CPU-resident marginal rows reached $1.001\times$ – $1.091\times$ the empty CPU reference. These rows expand the finite candidate family used by the theorem-facing replay; they are not used as evidence that the production scheduler already defaults to the optional global batch policy.

On the q01 GPU-heavy list, the node007 profile-1 measurement now lies on the measured service envelope, so the candidate and the table-goodput/delay external-policy rows all select profile 1 on the single-workload q01 replay. The implemented statewise logic is a lower-service no-upshift drain, not a hidden post-hoc rule: it cannot raise the base action and it only lowers a profile after service and waiting-backlog guards pass. The optional global-batch version improves q01 compute, q01 LLM, the q01 model portfolio, and the mixed portfolio relative to the current candidate. It leaves q01 CNN, q10, and q11 non-regressed under the 0.5% replay tolerance. The paper therefore treats global batching as an algorithmic upgrade path rather than as a default production-dispatch claim.

7.3. Theorem-condition and live-sanity checks

The hardware-local lower-service experiment instantiates the slack inequality in Theorem 1 separately on each of four GPU execution classes. For each class, the selected one-sided lower confidence bound (LCB) of a dedicated action is used as that workload's diagonal service scale. The four normalized workload coordinates therefore have unit dedicated service, and an equal stationary mix

Table 11 Hardware-local lower-service stability certificates.

Node	Execution class	δ	η	LCB coverage
Pending final export	–	–	–	–

Each row uses four normalized Bernoulli arrival streams at load fraction 0.8. LCB coverage is nominal per node; cross-node simultaneous coverage uses the reported union-bound certificate. These rows certify exact measured dedicated actions. Loaded co-location actions remain statewise replay evidence rather than being silently absorbed into this empty-state stochastic model.

supplies 1/4 normalized work units per slot to each coordinate. At load fraction 0.8, the registered Bernoulli model has

$$\begin{aligned}\delta &= \frac{1}{4} - \frac{0.8}{4} = 0.05, \\ \eta &= \delta - (L\rho + \epsilon_{\text{est}} + \beta + \alpha_1) \\ &= 0.000000.\end{aligned}\tag{48}$$

Here the candidate family is the exact finite set of measured dedicated actions, service is already the admitted one-sided LCB, and the finite oracle is exact and unpenalized. Thus the certificate records $L\rho = \epsilon_{\text{est}} = \beta = \alpha_1 = 0$ relative to that lower-service action family; it does not assert zero mean-service estimation error. The submission gate requires the exported minimum 0.000000 to be strictly positive. Table 11 is generated from the same cache hash as the replay. Its four rows are separate hardware-local stochastic models and do not pool nominally identical hosts.

The final service cache does not infer job completion time from a short steady-state window. Every admitted GPU row contains a task-native natural-completion model with separate initialization, periodic work, and final checkpoint/output phases; no history-fallback row is admitted. Empty and loaded rows are keyed by physical execution class, workload, profile, and directional resident/target state. Consequently, rates from node007 are not borrowed for either jtl110 host or jtl311linux, and loaded target rates cannot overwrite empty-state legacy indices.

The same cache also re-evaluates the controlled migration ledger. Across 9 registered pure-CPU, pure-GPU, and hybrid-RL counterfactuals, migration is selected in 0 cases. The geometric ratios of without-migration to with-migration completion cost are 1.000000× for makespan and 1.000000× for mean flow. These ratios are reported even when they equal or fall below one: the admissibility rule charges checkpoint flush, synchronization, environment staging, resume warmup, lost work, and failure risk before selecting a move. Ordinary running user jobs are not migration candidates.

We also ran a corner-case marginal-efficiency experiment while the cluster was idle. Each probe used benchmark logs with explicit progress rate and estimated time remaining (ETA) lines. After three stable windows the probe stopped and the measured service row was available for replay.

Table 12 Controlled corner-case marginal service probes.

Scenario	Host bucket	Resident rate	Marginal rate	Loaded/empty ratio
CPU empty + small CPU	node001 CPU host	—	17.504	—
CPU 96-worker resident + small CPU	node001 CPU host	33.573	17.354	0.991
CPU 180-worker resident + small CPU	node001 CPU host	20.337	17.383	0.993
GPU empty + small CUDA	node007 GPU host	—	491.827	—
30.0GB GPU resident + small CUDA	node007 GPU host	1049.791	443.983	0.903

Rates are benchmark service units per second from explicit progress logs. The loaded/empty ratio compares the marginal row with the same small-task empty-resource baseline. The table is a controlled synthetic marginal-service slice, not a full-stack state-of-the-art performance claim or a production-wide theorem population.

Table 13 Gavel-style resident-delay/JCT holdout on the controlled co-location rows.

Scenario	Co-run mean JCT	Defer mean JCT	Mean JCT reduction	Makespan reduction
CPU 96-worker resident + small CPU	0.044	0.053	17.0%	29.1%
CPU 180-worker resident + small CPU	0.053	0.054	1.4%	30.4%
30.0GB GPU resident + small CUDA	0.217	0.322	32.5%	46.8%

Seconds are measured benchmark service-window seconds. The defer baseline uses the fastest observed resident-alone progress window, which favors defer. The table is a scoped measured-service holdout, not a direct Gavel full-stack execution.

Table 12 reports the last stable marginal rate and the ratio to the corresponding empty resource baseline. The 30GB row holds a four-GPU resident memory job on the direct node007 GPU host and then adds a small CUDA task on one card. The CPU rows run 96-worker and 180-worker resident jobs on 192-core nodes and then add a small CPU task. These rows show that high resident video memory does not by itself imply a low marginal CUDA rate on this slice; on the idle 192-core CPU node, the single-thread marginal CPU micro-kernel also retained its empty-node rate under the 96-worker and 180-worker resident loads used here. Scheduleurm admits all stable positive-service rows in this controlled slice. Memory-packing policies admit the GPU rows when memory fits, and goodput policies admit rows whose loaded-to-empty ratio exceeds 0.90. The five stable canonical rows also have positive row-level lower-service slack at an offered load of 0.8 times the measured lower-service rate. This makes the corner-case rows theorem-facing for this finite measured slice without changing the default replay cache. Table 13 reports the same rows using job completion time (JCT) rather than marginal service rate.

A controlled live-dispatch check ran q01/q11 ScheduleurmBench tasks through the optional robust lower-service hook. A 6-task run emitted 7 theorem-grade dispatch slots and 13 candidate rows. A larger 32-task run on an available two-GPU node emitted 32 theorem-grade slots and 56 candidate rows, completed all 32 tasks, and had $\alpha_0 = \alpha_1 = 0$. These runs check that the hook can launch and complete admitted tasks. They are separate from the larger production-history evidence.

Table 14 Controlled and production evidence populations.

Population	Evidence	Interpretation
Controlled 6-task launch	6 launched and completed tasks; 7 theorem slots; 13 candidate rows.	Hook-level launch/completion check for admitted candidate rows.
Controlled 32-task launch	32 launched and completed tasks; 32 theorem slots; 56 candidate rows; $\alpha_0 = \alpha_1 = 0$.	Larger controlled live check on a two-GPU node.
Strict scheduler-history production	4,058 admitted launches; 4,047 completions; 35 domains; 13 nodes; no strict unadmitted launched rows.	Production-history evidence for admitted rows; raw 30-day telemetry is not the theorem population.
Completed-active service-map bridge	3,437 strict mapped records out of 3,437; mapped-slice slack 9.123×10^{-6} ; service-map oracle bridge with $\alpha_0 = \alpha_1 = 0$.	Lower-service bridge for the controlled completed-active population.
Rolling queue snapshots	Active-production progress and closed shadow-subset oracle checks when available; CPU fallback slots excluded from the theorem subset.	Read-only monitoring and oracle conversion, not raw-history closure.
Live-oracle boundary	Large-scale live-oracle trace fields and global-batch soft-hint hook exist, but production-default global dispatch is not claimed.	Separates hook readiness from deployed theorem-grade production scheduling.

The populations are kept separate because they support different claims: controlled live execution, strict admitted-history completion, service-map bridging, read-only monitoring, and live-oracle readiness.

The production evidence is based on strict scheduler history rather than on raw telemetry. Rows outside scheduler control, including attempted-only, auto-adopted, diagnostic, and read-only probe rows, are excluded. The admitted history contains 4,058 strict theorem-facing launches, 4,047 completions, 35 workload domains, 13 nodes, and no strict unadmitted launched rows. For closed shadow-subset snapshots, the theorem bridge has $\alpha_0 = \alpha_1 = 0$; CPU fallback slots use legacy sort-key semantics and are excluded from the theorem subset. The live evidence therefore supports controlled completion, strict scheduler-history completion, and read-only multi-node Scheduleum coverage. It does not convert raw telemetry or future unknown jobs into theorem-facing arrivals. Table 14 summarizes the four evidence populations.

7.4. Finite-trajectory generalization beyond compute

The server action can be generalized without erasing domain physics by treating an action as a complete finite configuration trajectory. A trajectory records placement or mode choices, admissible reconfiguration decisions, cumulative lower service, duration, and a bounded penalty. In each experiment below, the same selector exactly maximizes generated-family robust score

$$a_n \in \arg \max_{a \in \mathcal{A}_n^{\text{traj}}} \left\{ \frac{(Q^F(n))^T \underline{S}_n^F(a) - P_n^F(a)}{\tau_n(a)} \right\}, \quad (49)$$

Here n indexes decision frames, $Q^F(n)$ is the frame-boundary backlog or urgency vector, $\mathcal{A}_n^{\text{traj}}$ is the finite generated trajectory family, $\underline{S}_n^F(a)$ is the cumulative lower-service vector, $P_n^F(a)$ is the cumulative bounded trajectory penalty, and $\tau_n(a) > 0$ is the trajectory duration. For the FJSP

and MMRCPSp renewal experiments, the artifact uses the actual duration ratio $\tau_n(a)/H_n$, where the positive normalization constant H_n is common to the frame. For each registered port context, every completed trajectory is instead idle-padded to a candidate-independent common horizon H_n ; division by that common horizon preserves the scalar ordering. The duration factor is essential in the former case: maximizing the unnormalized cumulative numerator is generally not equivalent when candidate trajectories have different lengths. The port terminal coordinates are bounded virtual cost service, not physical vessel departures. The queue/urgency coordinates and feasibility oracle are domain-specific. Proposition 3 supplies the variable-duration stochastic interface, and the Lean theorem `frameApproximateOracle_iff_durationNormalized` checks the corresponding cumulative-oracle/rate-oracle equivalence for positive duration. The deterministic holdouts in this subsection test the candidate/oracle/penalty layer and do not by themselves instantiate a stochastic arrival model.

The latest confirmation protocols augment, rather than overwrite, the earlier feasibility holdouts. The FJSP confirmation selects 20 previously unused instances from the Hurink edata, rdata, sdata, and vdata families (Hurink et al. 1994). For each instance, the robust selector and five fixed dispatch rules are compared using exact integer makespan and sum-completion ledgers. The fixed rules are first-in-first-out (FIFO), earliest completion time (ECT), shortest processing time (SPT), most operations remaining (MOR), and most work remaining (MWKR). A separate five-second, single-worker OR-Tools 9.14 Constraint Programming with Boolean Satisfiability (CP-SAT) run provides a makespan reference (Perron et al. 2023). Its sum-completion coordinate is descriptive because CP-SAT optimizes makespan only, and a feasible status is not an optimality proof.

The prospective structural MMRCPSp holdout registers 25 PSPLIB instances, five each from j12, j14, j16, j18, and j20 (Kolisch and Sprecher 1997). The selected candidate is Pareto nondominated against the frozen heuristic family on all 25 instances, strictly dominates every heuristic on 11, and attains the public makespan optimum on 7; its geometric-mean makespan ratio to the public optimum is 1.1667. These figures do not imply global or universal superiority. The subsequent stochastic experiment reuses the resulting known 25-class action library, where one action completes one project from one registered class. Seven exogenous unit-time arrival scenarios are replayed against duration-normalized robust MaxWeight, unnormalized robust MaxWeight, shortest registered trajectory, longest queue then shortest trajectory, and round-robin class selection. The first stream matrix uses three seeds; a later class-balance protocol freezes five new arrival tapes. The class-balance metric family was registered after observing the first matrix but before observing

those new tapes. It is therefore a prospective arrival-tape check on a known instance and action library, not a new structural-instance holdout.

The port evidence keeps physical source semantics and synthetic migration semantics non-substitutable. The BACASP-S source-core certificate evaluates a 30-vessel berth/quay instance with crane travel and setup times (Correcher et al. 2024); it contains no mid-service migration. Three synthetic event-driven instances add berth, quay-crane, yard-block, and gate-lane resources together with re-berthing, crane reassignment, and yard rehandle. One locked 31-trajectory family is evaluated over the public and three synthetic contexts, giving 124 feasible trajectory-context ledgers and zero generated-family oracle gap. The globally selected plan is Pareto nondominated on the public source-core comparison and all three synthetic comparisons. This current-code refresh is a reproducible registered replay, not a prospective holdout. A separate frame certificate verifies that all 31 trajectories fit each context's positive candidate-independent horizon and that the referenced Lean frame file compiles. Because the selected terminal coordinates are virtual costs, this does not provide a stochastic physical-service or positive-recurrence certificate. Table 15 reports the resulting mixed evidence.

Two negative chains are especially informative. First, the earlier MMRCPSP instance-feasibility holdout exposed a renewable-resource error in the nonrenewable reserve dynamic program; its opened-row regression and disjoint repair holdout remain archived. Second, the new renewal-stream result shows that a valid exact MaxWeight oracle need not minimize aggregate delay: the shortest-trajectory baseline wins every aggregate-queue cell. The prospective class-balance result instead establishes a 35/35 nondominated tradeoff, not strict superiority. This is consistent with the mathematics: the renewal-frame theorem controls drift under slack assumptions and is not a theorem of makespan or total-delay optimality.

The FJSP candidate ledgers and all qualitative CP-SAT relations reproduce on a second run. Three exact CP-SAT gaps vary because a five-second wall-clock budget can return different incumbents; this variation is reported rather than treated as candidate-ledger drift. Both MMRCPSP stream matrices reproduce exactly after decompression. Their raw gzip hashes differ only because the gzip header stores the output basename. The refreshed port ledger removes only observed wall-clock runtime fields and uses a fixed gzip header; independent runs are therefore byte-identical as well as semantically identical. The port source-core and synthetic migration semantics remain separate: the former contains BACASP-S data but no migration, whereas the latter contains migration semantics but no physical terminal observations. These boundaries prevent portability evidence from becoming a domain-wide state-of-the-art claim.

Table 15 Cross-domain finite-trajectory evidence.

Domain	Frozen protocol	Candidate result	Claim boundary
FJSP	20 unused Hurink instances; six registered policies; independent five-second, one-worker CP-SAT reference	Robust selector nondominated 20/20 against fixed rules. CP-SAT feasible 20/20, optimal status 10/20, and dominates the selected ledger 14/20. Qualitative relations reproduce 20/20; exact numeric gaps reproduce 17/20.	CP-SAT is makespan-only and wall-clock limited; no global optimum or arbitrary-solver claim.
MMRCPSP structural holdout	25 prospective PSPLIB instances across j12, j14, j16, j18, and j20	Candidate nondominated 25/25 against frozen heuristics; strictly dominates every heuristic 11/25; attains public makespan optimum 7/25; geometric-mean makespan/optimum 1.1667.	Public optimum and CP-SAT reference are makespan-only; no global or universal superiority claim.
MMRCPSP aggregate queue	Known 25-class action library; seven scenarios, three seeds, and five policies; 105 runs	Exact robust oracle 21/21 scenario/seed cells, but shortest trajectory dominates it 21/21 on final backlog and time-average total queue.	Retained negative result; no total-delay superiority or recurrence claim.
MMRCPSP class balance	Same known library; seven scenarios, five new seeds, and five policies; 175 runs	Exact robust oracle and Pareto nondominance 35/35 on total time-average queue, worst-class time-average queue, and worst-class final backlog. Strictly better than every baseline 0/35; baseline dominance 0/35.	New arrival tapes, not new structural instances; no universal performance claim.
Port	30-vessel BACASP-S source core; three registered four-resource synthetic instances; one locked 31-trajectory family	All 124 trajectory-context pairs are feasible with generated-family $\alpha_0 = \alpha_1 = 0$; selected global plan is Pareto nondominated on the public context and 3/3 synthetic contexts. The earlier statewise source-core robust policy remains dominated by reconfiguration-greedy and SPT.	Registered replay, not prospective holdout; virtual-cost frame bridge is compiled, but no physical migration observation, stochastic port slack, industrial deployment, or global port optimum.

Every comparison is scoped to its registered finite policy family and stated cost coordinates. The rows deliberately retain comparator wins and do not assert domain-wide superiority.

7.5. Formal and production artifacts

The Lean proof artifact is submitted as a supplement. It contains the consolidated proof file, the Lean toolchain specification, the build log, a paper-theorem to Lean-theorem crosswalk, and checksums for the submitted files. The supplement build passes, and a comment-aware proof search finds no `sorry`, `admit`, or `axiom` proof terms. Full file hashes are reported in the supplement manifest rather than in the manuscript text.

For production data, the current theorem-facing population is the completed-active `Scheduleurm`-controlled set. It has 3,437 strict mapped records out of 3,437, with mapped-slice slack 9.123×10^{-6} . The service-map oracle bridge passes with $\alpha_0 = \alpha_1 = 0$. By contrast, the raw 30-day history has 6,957 records, with 4,825 mapped records and 2,132 unmapped records, so it is not a clean theorem population. This distinction is important: the artifact proves coverage of a controlled completed-active population, not closure of every raw, attempted-only, external, or future scheduler row.

8. Discussion and Limitations

This paper connects a queueing-stability theorem with the objects available in a real heterogeneous scheduler. The main mathematical point is that candidate restriction is not treated as a heuristic

shortcut. It is represented as an explicit support-function loss and paid from capacity slack together with estimation, penalty, and oracle losses. This gives a readable accounting rule: a scheduler may use a finite candidate family only when the remaining drift margin is still positive.

The finite-cell construction strengthens this accounting from a purely assumption-driven statement to a candidate-generation result. A finite feature partition controls candidate cardinality, while representative radius and service sensitivity control the induced support loss. This separates computational complexity from statistical calibration: the partition determines how many actions the oracle must inspect, whereas perturbation data determine how much capacity slack the compression consumes. The diagonal LCB result performs the analogous separation across heterogeneous service units, and the finite-horizon corollary converts residual slack into an expected-backlog bound rather than only a recurrence statement.

The Scheduleurm case study shows how this accounting can be implemented. The scheduler records measured lower-service rows, admissible configuration actions, capacity boundaries, penalties, oracle gaps, and admission status. The same records support four roles: performance replay, stability-condition calibration, production-history classification, and optional live-hook checks. This separation is useful because it prevents a strong empirical result in one population from being silently transferred to another.

The strongest empirical statement remains a measured finite-slice statement. For the declared service domains, exact measured actions give $\rho = 0$, and compressed candidate families report the corresponding nonzero $L\rho$. A future task is theorem-facing only if it is admitted to a measured domain with a positive lower-service row; otherwise it is assigned to a probe/defer state with zero theorem service. This is sufficient for the present measured-domain certificate, but it is not a universal positive-service theorem for all future workloads and hardware states.

The external-scheduler comparisons have a similarly precise role. They test whether the measured candidate family omits a useful scheduling semantic, such as goodput allocation, delay priority, or interference guarding. Once such a semantic is represented by a lower-service row and a bounded penalty, it becomes another finite action in the robust selector. This is the right comparison for the candidate-set theorem. It is narrower than a claim that Scheduleurm has beaten every external scheduler in its native full-stack deployment.

The production evidence should also be read at the correct level. Controlled live launches show that admitted theorem-hook actions can run to completion. Strict scheduler history shows that admitted production rows can be classified and completed at scale. Read-only shadow traces show

that current queue states can be converted into theorem-facing candidate rows without mutating the queue. Together these checks support the measured *Scheduleurm* certificate. They do not turn raw telemetry, future unknown arrivals, or every live dispatch into a stability-certified arrival stream.

Three technical directions remain open. The first is broader fabric-cover calibration: extending the positive-service certificate beyond the declared finite domains requires perturbation evidence for the feature map, projection, cover radius, Lipschitz envelope, and statistical failure probability. The second is online learning: the active-bucket sampler and regime detector have formal conditional guarantees and replay checks, but they should be validated in live A/B experiments before becoming default scheduling logic. The third is migration-aware control. Proposition 3 closes the mathematical variable-duration boundary: setup time, lost work, and resume risk enter a cumulative net-service frame or a unit-matched bounded penalty, and a positive minimum duration transfers per-time slack to embedded-chain drift. The remaining obligation is operational rather than algebraic. The current controlled migration rows estimate checkpoint, synchronization, staging, and resume components, but they have not yet been assembled into a domain-wide stochastic frame model with conditional cumulative-service and moment certificates. We therefore do not infer a semi-Markov stability claim from the deterministic migration or cross-domain trajectory holdouts.

9. Conclusion

We studied heterogeneous compute scheduling as a configuration-action stochastic processing network. The resulting robust candidate *MaxWeight* theorem gives an explicit slack budget for candidate approximation, lower-service estimation, bounded penalties, and approximate optimization. If the residual margin is positive, the policy satisfies finite-horizon expected-backlog bounds and a finite-set Foster-Lyapunov recurrence certificate. A finite feature-cell construction bounds both candidate cardinality and the resulting $L\rho$ support loss, while diagonal scaling preserves the meaning of lower-confidence service across heterogeneous workload units. A separate variable-duration frame result extends the same slack accounting to cumulative configuration trajectories and recovers the slotted theorem at unit duration. The fixed-family, queue-indexed statewise, candidate-construction, diagonal-scaling, and frame statements are formalized in Lean.

Scheduleurm provides a concrete instantiation of the theorem. Its measured finite slices close the lower-service stability conditions for declared workload domains. On 24 closed-batch and 192 online replay comparisons, the robust candidate improves legacy completion metrics and its trace-level relationship to every registered policy-semantics baseline is reported on the same measured

service cache. Controlled live launches and strict scheduler-history evidence show how the theorem-facing objects can be produced by an operational scheduler. Frozen finite-trajectory experiments on FJSP, MMRCPS, and port models further test the candidate/oracle/penalty interface. Their mixed results include independent-solver and registered-policy counterexamples, so they support portability of the interface without claiming domain-wide optimality or automatic transfer of the server stochastic model.

The contribution is therefore not a universal claim about every future workload or every external scheduler implementation. It is a mathematical and empirical template for making approximate configuration-action scheduling certifiable: define the action family, measure conservative service, account for every loss, and claim stability only when the residual slack remains positive.

Appendix: Conventional Proof Details

This appendix gives the conventional, paper-readable proof chain behind Theorem 1. It is intentionally redundant with the main text: the purpose is to make clear which part is mathematics, which part is a measured Schedulerm certificate, and which part is checked by Lean. The Lean artifact is a formal check of the algebraic and recurrence implications; it is not a substitute for empirical certificates of service rows, lower confidence bounds, candidate admission, oracle gaps, or moment bounds.

EC.1. Capacity Slack as a Support-Function Margin

Let $\mu(a) \in \mathbb{R}_+^{|I|}$ denote the true mean service vector of full action $a \in \mathcal{A}^{\text{full}}$, and let $H_{\mathcal{A}^{\text{full}}}(q) = \max_{a \in \mathcal{A}^{\text{full}}} q^\top \mu(a)$ be the full-action support function for nonnegative backlog weights $q \in \mathbb{R}_+^{|I|}$. A stationary randomized policy over full actions is a probability vector $x \in \text{Mix}(\mathcal{A}^{\text{full}})$. If it has coordinate slack $\delta > 0$ against the arrival mean λ , then

$$\lambda_i + \delta \leq \sum_{a \in \mathcal{A}^{\text{full}}} x_a \mu_i(a), \quad i \in I. \quad (50)$$

Multiplying each coordinate by $q_i \geq 0$ and summing gives

$$q^\top \lambda + \delta \|q\|_1 \leq \sum_{a \in \mathcal{A}^{\text{full}}} x_a q^\top \mu(a). \quad (51)$$

Because x is a convex combination, the right-hand side is at most the maximum full-action support:

$$\sum_{a \in \mathcal{A}^{\text{full}}} x_a q^\top \mu(a) \leq H_{\mathcal{A}^{\text{full}}}(q). \quad (52)$$

Hence full-action capacity slack implies

$$q^\top \lambda + \delta \|q\|_1 \leq H_{\mathcal{A}^{\text{full}}}(q). \quad (53)$$

This is the only place where the capacity-region hypothesis enters the drift proof. It converts an interior capacity statement into a MaxWeight support margin that can be compared to candidate actions. The paper uses the support-function statement as Assumption 1; the operational necessity direction is separate and requires a load-certified stochastic model and a conservation law.

EC.2. Candidate Fabric Cover and Support Loss

The scheduler cannot enumerate every full configuration. It evaluates an admitted candidate family $\mathcal{A}_t^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$, possibly depending on the observed state t . The finite-candidate theorem needs a support loss certificate: for every $q \geq 0$,

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1. \quad (54)$$

When the loss is obtained from a calibrated fabric metric, let $\Phi(a)$ be the finite feature representation of action a , let d_Φ be the feature metric, and suppose each full action a has a projected candidate $\pi_t(a) \in \mathcal{A}_t^{\text{cand}}$ satisfying

$$d_\Phi(a, \pi_t(a)) \leq \rho. \quad (55)$$

If service is Lipschitz in this representation,

$$\|\mu(a) - \mu(\pi_t(a))\|_\infty \leq L d_\Phi(a, \pi_t(a)), \quad (56)$$

then for any maximizer $a^\star \in \arg \max_{a \in \mathcal{A}^{\text{full}}} q^\top \mu(a)$,

$$\begin{aligned} H_{\mathcal{A}^{\text{full}}}(q) &= q^\top \mu(a^\star) \\ &\leq q^\top \mu(\pi_t(a^\star)) + \|q\|_1 L\rho \\ &\leq H_{\mathcal{A}_t^{\text{cand}}}(q) + L\rho \|q\|_1. \end{aligned} \quad (57)$$

Thus $\epsilon_{\text{cand}} = L\rho$ in the fabric-cover case. In measured finite slices where the candidate support is certified directly, the artifact uses the observed support gap itself as ϵ_{cand} ; when the candidate set is exact on the measured slice, this term is zero. This is why the proof does not shrink the action space informally: every restriction must be paid for by the explicit support-function loss in (54).

Proposition 1 supplies a constructive version of this step. It forms candidates as representatives of active feature cells, bounds the candidate cardinality by the number of cells, and derives the same $L\rho$ support loss. Thus the theorem can consume either a directly audited support gap or a finite-cell construction certificate; only the latter supports a claim of nontrivial action-space compression.

EC.3. Lower-Service Rows, Penalties, and Approximate Oracles

The robust scheduler optimizes conservative service, not the unknown true mean service. For each admitted candidate $a \in \mathcal{A}_t^{\text{cand}}$, the artifact provides a lower-service vector $\mu_t^L(a)$. The lower-service support loss is

$$H_{\mathcal{A}_t^{\text{cand}}}(q) \leq \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^L(a) + \epsilon_{\text{est}} \|q\|_1. \quad (58)$$

The scheduling score is

$$\Psi_t(a) = Q^\top \mu_t^L(a) - K_t(a) - G_t(a), \quad (59)$$

where $K_t(a)$ and $G_t(a)$ are bounded switching, risk, or guard penalties. Equation (58) is the lower-confidence-bound bridge from measured service rows to the drift proof. The coordinate-scaled sufficient condition for this bridge is given in Proposition 2: on the simultaneous confidence event it charges $2\epsilon_{\text{LCB}} \sum_i s_i q_i$, or $2\epsilon_{\text{LCB}} C_s \|q\|_1$ under a uniform scale bound. This prevents a high-throughput workload from defining the absolute error unit for all other classes. The selected action a_t need not be the exact maximizer. It satisfies

$$\max_{a \in \mathcal{A}_t^{\text{cand}}} \Psi_t(a) - \Psi_t(a_t) \leq \alpha_0 + \alpha_1 \|Q\|_1, \quad (60)$$

and all admitted candidate penalties obey

$$0 \leq K_t(a) + G_t(a) \leq P_0 + \beta \|Q\|_1, \quad a \in \mathcal{A}_t^{\text{cand}}. \quad (61)$$

Combining the capacity support bound (53), the two support-loss bounds (54) and (58), the oracle gap (60), and the penalty bound (61) yields the selected lower-service margin. First,

$$\max_{a \in \mathcal{A}_t^{\text{cand}}} Q^\top \mu_t^L(a) \geq Q^\top \lambda + (\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}}) \|Q\|_1. \quad (62)$$

Second, by the oracle gap and the all-candidate penalty bound,

$$\begin{aligned} Q^\top \mu_t^L(a_t) &\geq \max_{a \in \mathcal{A}_t^{\text{cand}}} Q^\top \mu_t^L(a) \\ &\quad - (P_0 + \alpha_0) - (\beta + \alpha_1) \|Q\|_1 \\ &\geq Q^\top \lambda + \eta \|Q\|_1 - (P_0 + \alpha_0), \end{aligned} \quad (63)$$

where

$$\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1). \quad (64)$$

The theorem therefore has a simple accounting rule: candidate loss, lower-service estimation loss, queue-scaled penalty, and queue-scaled oracle loss all consume the same capacity slack. A stability certificate requires $\eta > 0$. Additive terms P_0 and α_0 do not change the slope of the drift; they enlarge the finite set outside which the drift is negative.

EC.4. Quadratic Drift and the Foster Certificate

The queue update is

$$Q_i(t+1) = [Q_i(t) - S_i(t)]^+ + A_i(t), \quad (65)$$

where $A_i(t)$ is arrival and $S_i(t)$ is potential service for class i . Completed departure is $D_i(t) = \min\{Q_i(t), S_i(t)\}$. The displayed update remains valid because unused potential service is removed by the positive part. For $V(Q) = \frac{1}{2} \sum_i Q_i^2$, the standard one-step inequality gives

$$\begin{aligned} \Delta V(t) &\leq \frac{1}{2} \sum_i (A_i(t)^2 + S_i(t)^2) \\ &\quad + Q(t)^\top A(t) - Q(t)^\top S(t). \end{aligned} \quad (66)$$

Assumption 4 supplies the conditional second-order bound

$$\mathbb{E} \left[\frac{1}{2} \sum_i (A_i(t)^2 + S_i(t)^2) \middle| \mathcal{F}_t \right] \leq B. \quad (67)$$

It also ties the stochastic process to the theorem-facing service rows:

$$\mathbb{E}[A(t) \mid \mathcal{F}_t] \leq \lambda, \quad (68)$$

$$\mathbb{E}[S(t) \mid \mathcal{F}_t] \geq \mu_t^L(a_t) \quad (69)$$

coordinatewise. Taking conditional expectation in (66) and using (63),

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] &\leq B + Q(t)^\top \lambda - Q(t)^\top \mu_t^L(a_t) \\ &\leq B + P_0 + \alpha_0 - \eta \|Q(t)\|_1. \end{aligned} \quad (70)$$

If N and $\gamma > 0$ satisfy

$$B + P_0 + \alpha_0 + \gamma \leq \eta N, \quad (71)$$

then whenever $\|Q(t)\|_1 > N$, (70) implies

$$\mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] \leq -\gamma. \quad (72)$$

This is the finite-set drift certificate used in Theorem 1. To translate the certificate into positive recurrence for a Markov chain, the paper also assumes a countable augmented state space, a finite nonempty backlog sublevel set, and a closed communicating class or equivalent local-return condition. Without those recurrence-side conditions the theorem should be read as a negative-drift certificate, not as an unconditional positive-recurrence statement for every possible implementation.

EC.5. Statewise Feasible Families and Operational Boundaries

The statewise corollary is a conditioning argument rather than a new drift identity. Let $\chi(t)$ record scheduler-visible information such as alive nodes, pinned tasks, queued jobs, and measured admission status. If the full family, candidate family, true and lower-service maps, feature map, penalty bounds, oracle bounds, and moment bounds are indexed by $X(t) = (Q(t), \chi(t))$ and satisfy the same constants for every admitted state, then the proof in EC.1–EC.4 holds after conditioning on $\mathcal{F}_t$. Uniform constants preserve both the drift and finite-horizon backlog bounds. Positive recurrence additionally requires the augmented-state conditions in Assumption 5; it does not follow merely by averaging over $\chi(t)$.

The operational capacity statement is kept separate. Sufficiency follows from the positive-slack drift chain above once a stochastic model certifies the arrival mean, service accounting, lower-service domination, moment bounds, and local recurrence conditions. Necessity is not obtained by defining a model to “stabilize” a load; it requires the model to encode the load and obey a conservation law showing that long-run departures cannot exceed the capacity base. This separation is essential for the paper’s claim boundary: Lean checks the implication from certified assumptions to drift and recurrence, while the experiments and admission checks certify when Scheduleum rows satisfy those assumptions.

EC.6. Variable-Duration Configuration Trajectories

The slotted theorem does not silently cover migration, machine reconfiguration, or port moves whose durations differ. We therefore state the separate frame result used by the trajectory interface. Let n index decision frames, let $\mathcal{F}_n^F$ be the information available at the start of frame n , and let the $\mathcal{F}_n^F$ -measurable duration satisfy

$$0 < \tau_{\min} \leq \tau_n \leq \tau_{\max} < \infty. \quad (73)$$

The frame-boundary queue obeys

$$Q^F(n+1) = [Q^F(n) - S^F(n)]^+ + A^F(n), \quad (74)$$

where $A^F(n)$ and $S^F(n)$ are cumulative arrivals and potential service during the frame. Define

$$\bar{S}^F(n) = \mathbb{E}[S^F(n) \mid \mathcal{F}_n^F]. \quad (75)$$

Suppose the conditional arrival and second-order bounds are

$$\mathbb{E}[A_i^F(n) \mid \mathcal{F}_n^F] \leq \tau_n \lambda_i, \quad (76)$$

$$\mathbb{E}\left[\frac{1}{2} \sum_i \{(A_i^F(n))^2 + (S_i^F(n))^2\} \middle| \mathcal{F}_n^F\right] \leq B_F. \quad (77)$$

Let ϵ_F combine candidate-support and conservative-service losses in rate units. The cumulative approximate-oracle and penalty obligations are

$$\begin{aligned} \tau_n H_{\mathcal{A}^{\text{full}}}(Q^F(n)) &\leq (Q^F(n))^T \bar{S}^F(n) \\ &\quad + \tau_n (\epsilon_F + \alpha_1) \|Q^F(n)\|_1 \\ &\quad + \alpha_0 + P_n^F. \end{aligned} \quad (78)$$

$$P_n^F \leq P_0 + \tau_n \beta \|Q^F(n)\|_1. \quad (79)$$

All quantities in (78)–(79) are in cumulative work units over the same frame; in particular, migration lost work cannot be inserted as an unmatched wall-clock scalar. For the controlled migration rows, let $W_n^{\text{rem}} > 0$ be one task's remaining work, let $\underline{\mu}_n^{\text{old}}$ and $\underline{\mu}_n^{\text{new}}$ be its per-task lower-service rates in the measured source and destination states, and let T_n^{mig} be the measured checkpoint, synchronization, staging, warmup, lost-work, and time-equivalent risk overhead. The migration trajectory is represented by the duration-normalized lower service

$$\underline{\mu}_n^{\text{mig,eff}} = \frac{W_n^{\text{rem}}}{T_n^{\text{mig}} + W_n^{\text{rem}} / \underline{\mu}_n^{\text{new}}}. \quad (80)$$

It enters the candidate family only when

$$\frac{W_n^{\text{rem}}}{\underline{\mu}_n^{\text{old}}} - \frac{W_n^{\text{rem}}}{\underline{\mu}_n^{\text{new}}} > T_n^{\text{mig}}, \quad (81)$$

equivalently $\underline{\mu}_n^{\text{mig,eff}} > \underline{\mu}_n^{\text{old}}$. Thus wall-clock overhead first changes the trajectory duration; it is not subtracted directly from a service-rate score.

PROPOSITION 3 (Variable-duration robust frame drift). *Suppose (73)–(79) hold and the full family has support slack δ . Define*

$$\eta_F = \delta - \epsilon_F - \alpha_1 - \beta. \quad (82)$$

Then

$$\begin{aligned} \mathbb{E} \left[V(Q^F(n+1)) - V(Q^F(n)) \middle| \mathcal{F}_n^F \right] \\ \leq B_F + \alpha_0 + P_0 - \tau_n \eta_F \|Q^F(n)\|_1 \\ \leq B_F + \alpha_0 + P_0 - \tau_{\min} \eta_F \|Q^F(n)\|_1, \end{aligned} \quad (83)$$

provided $\eta_F > 0$. If the frame-boundary augmented state is countable, its backlog sublevel sets are finite and nonempty, and the usual local-return condition holds, then any $N \in \mathbb{N}$ and $\gamma > 0$ satisfying

$$B_F + \alpha_0 + P_0 + \gamma \leq \tau_{\min} \eta_F N \quad (84)$$

give a finite-set Foster recurrence certificate for the embedded frame-boundary chain. Moreover, elapsed physical time through frame n is at most $n\tau_{\max}$.

Proof. The quadratic queue inequality applied to (74), followed by conditional expectation, gives

$$\begin{aligned} \mathbb{E}[\Delta_F V(n) \mid \mathcal{F}_n^F] &\leq B_F + \tau_n \left(Q^F(n)\right)^\top \lambda \\ &\quad - \left(Q^F(n)\right)^\top \bar{S}^F(n), \end{aligned} \quad (85)$$

where $\Delta_F V(n)$ denotes the frame-to-frame Lyapunov increment. Full-action support slack implies

$$\left(Q^F(n)\right)^\top \lambda + \delta \|Q^F(n)\|_1 \leq H_{\mathcal{A}^{\text{full}}} \left(Q^F(n)\right). \quad (86)$$

Multiplying (86) by τ_n , substituting (78) and (79) into (85), and collecting coefficients yields the first inequality in (83). Positivity of η_F and $\tau_n \geq \tau_{\min}$ gives the second. Equation (84) makes the conditional drift at most $-\gamma$ outside the finite sublevel set, so the same local-return argument as in EC.4 applies to the embedded chain. Finally, $\sum_{k=0}^{n-1} \tau_k \leq n\tau_{\max}$ proves the calendar-time bound.

If arrivals and service are rate-scaled throughout a frame and their unit-time second-order term is at most B , then the formal artifact also checks the constructive choice $B_F \leq \tau_{\max}^2 B$. Setting $\tau_n = \tau_{\min} = \tau_{\max} = 1$, $B_F = B$, and $\epsilon_F = \epsilon_{\text{cand}} + \epsilon_{\text{est}}$ recovers the slotted drift theorem. Thus the result strengthens rather than relabels the migration boundary: a measured migration row must still certify cumulative net service, duration, checkpoint/synchronization/resume loss, and risk before it can enter a stochastic frame model.

EC.7. Lean Crosswalk and Artifact Assumptions

Tables 16 and 17 separate the formal proof contract from the empirical Scheduleurm certificates. The manuscript uses paper-facing statement names; exact Lean identifiers are kept in the supplementary theorem crosswalk so that the article does not become a code index.

Table 16 Paper statements and Lean proof groups.

Paper statement	Lean proof group	What the formal result checks
Proposition 1	Finite feature-cell candidate construction theorem	Candidate containment, cardinality bounded by active feature cells, and nonzero $L\rho$ support loss.
Theorem 1, exact candidate version	Fixed-family exact-oracle robust candidate MaxWeight theorem	Support-function candidate loss, robust lower-service drift, and finite-set recurrence certificate.
Theorem 1, approximate-oracle version	Fixed-family approximate-oracle robust candidate MaxWeight theorem	Calibrated fabric cover, bounded second moment, additive and queue-scaled oracle loss, and slack accounting.
Corollary 2	State-indexed candidate-family theorem	State-dependent feasible families under the same uniform constants used in the fixed-family theorem.
Proposition 2	Diagonal-scaled lower confidence bound theorem	Heterogeneous service-unit scaling and lower-service support loss in normalized coordinates.
Corollary 1	Foster telescoping backlog theorem	Finite-horizon cumulative-backlog and asymptotic expected-backlog bounds from the residual drift margin.
Proposition 3	Variable-duration frame drift and embedded recurrence theorems	Cumulative-work oracle, bounded frame penalty, uniform frame drift, duration-one reduction, and elapsed-time bound.
Operational capacity boundary	Load-certified capacity sandwich theorem	Positive-slack sufficiency and zero-slack conservation-law necessity once the stochastic model encodes arrivals and service accounting.

The exact Lean theorem identifiers, file hashes, and build log are supplied in the supplementary artifact crosswalk. This table records the mathematical correspondence used by the paper.

Table 17 Formal hypotheses and Scheduleurm certificates.

Object	Formal role	Scheduleurm certificate or boundary
λ, δ	Arrival mean and full-action capacity slack.	Online arrival replay, load sweep, and finite lower-service capacity checks.
ϵ_{cand} or $L\rho$	Candidate-set or fabric-cover support loss.	Measured finite-slice support gaps, declared bucket admission, and fabric-cover calibration rows when available.
$\mu_t^L, \epsilon_{\text{est}}$	Conservative lower-service rows and estimation loss.	Holdout lower confidence bound checks, measured service cache, and admission/probe status.
P_0, β	Bounded and queue-scaled penalty consumption.	Penalty-unit records, guard/tie-break checks, and bounded switching configuration.
α_0, α_1	Additive and queue-scaled approximate oracle loss.	Oracle-gap check comparing selected action against theorem-facing candidate families.
B, N, γ	Second-order moment bound and finite-set drift threshold.	Workload bounds, controlled trace statistics, and recurrence-side finite-state admission conditions.
$\tau_{\min}, \tau_{\max}, B_F$	Frame-duration bounds and cumulative second-order term.	A trajectory row must use natural elapsed duration and cumulative arrivals/service; unit-rate scaling yields the audited bound $B_F \leq \tau_{\max}^2 B$ when applicable.
$\bar{S}^F(n), P_n^F$	Conditional cumulative service and frame-level migration/reconfiguration loss.	Checkpoint flush, synchronization, staging, resume warmup, lost work, and failure risk must be converted to the same cumulative service/penalty units; deterministic domain holdouts alone do not certify this stochastic bridge.
Statewise $\chi(t)$	Scheduler-visible feasible-family index.	Alive node, pinned-job, queued-job, service-cache, and admission-state trace fields.
Operational model certificate	Links stochastic arrivals and services to the queueing theorem.	Load-certified model contract plus conservation-law boundary; not inferred from Lean alone.

The table is deliberately conservative: Lean verifies the implications once these hypotheses hold, while the Scheduleurm artifact determines which measured rows satisfy them.

Appendix: Supplementary Evidence Index

The full machine-readable status dashboard is provided in the supplementary artifact package. Tables 18 and 19 give a compact index of what is included and what each evidence family is allowed to support. They summarize the detailed supplementary status dashboard in manuscript-facing evidence categories; the row-level dashboard remains in the supplement, so the compression changes presentation rather than the underlying evidence.

Table 18 Supplementary evidence index: theorem and Schedulerm certificates.

Gate family	Retained dashboard evidence	Boundary kept in manuscript
Formal proof package	Consolidated Lean upload, toolchain, build log, theorem crosswalk, no-sorry/admit/axiom proof search, and checksums.	Formalizes theorem implications only after the stated service, moment, slack, and recurrence assumptions are certified.
Declared finite-domain cover	q00/q01/q10/q11 service-cache summaries, admission boundaries, measured support gaps, all-state safety cover, and probe/defer status for unknown states.	Supports declared finite measured domains; it does not assert positive service for arbitrary future states.
Lower-service and slack certificates	Holdout lower confidence bound (LCB) rows, selected-profile stochastic LCB certificate, row-level lower-service slack, $L\rho$ accounting, and finite second-moment thresholds.	Certifies lower-service capacity for admitted rows; mean-service and broad fabric-cover claims require additional calibration.
Replay and online stress	Explicit task-list replay, Poisson and bursty load sweeps, ablations, guarded-policy comparisons, and slack accounting tables.	Compares policies on the same measured service cache; it is not a proof that a static replay trace is a stationary arrival process.
Controlled launch/completion	Six-task and 32-task theorem-admitted launches, completion records, candidate rows, and $\alpha_0 = \alpha_1 = 0$ oracle-gap checks.	Supports controlled hook execution; it is separate from production-wide organic dispatch claims.
Production-history bridge	Strict completed-active production classification, strict launched/completed counts, read-only shadow traces, service-map oracle bridge, and exclusion of attempted-only or external rows.	Supports admitted history evidence; raw telemetry and future unknown jobs are outside the theorem population.
Live-oracle and global-dispatch readiness	Optional global-batch soft-hint hook, live-oracle trace fields, active-production shadow rows, and production readiness bridge.	Shows hook readiness and read-only conversion to theorem rows; it does not make the global batch policy the default production dispatcher.

This table indexes evidence used for the main theorem, measured finite-slice claims, and Schedulerm production-history bridge.

Table 19 Supplementary evidence index: external-policy and extension checks.

Gate family	Retained dashboard evidence	Boundary kept in manuscript
Policy-semantics baseline universe	Gavel-, Pollux-, Sia-, IADeep-, Salus-, fairness-, packing-, short-job, and delay-style policy rows evaluated on the same measured service cache.	Tests whether the candidate family omits useful finite scheduling semantics; it is not direct binary/full-stack superiority.
Gavel calibration and holdout rows	Adapter certificate, profile-aware calibration, resident-delay/JCT holdout, native same-workload simulator row, and same-host physical probe.	Keeps scalar service-unit equivalence separate from measured-cache policy comparison and scoped runtime evidence.
Named external runtime probes	Same-host runtime or adapter probes for Gavel, Pollux/AdaptDL, Sia, IADeep, and Salus, with paired workload and environment metadata when available.	Diagnostic implementation evidence only; original multi-node control-plane superiority is not claimed.
External action-union and algorithm upgrades	External policy-family actions, trajectory/action-semantics union rows, adaptive scalarized replay, state-dependent marginal lookup, global lookahead, and ETA reuse.	Shows the candidate family can absorb registered semantics; it is not production-default launch evidence.
Controlled migration-cost probes	Physical checkpoint, sync, and resume timing rows for controlled CNN, hybrid-RL, and CPU-surrogate payloads at 25%, 50%, and 75% progress.	Calibrates K_{mig} for benchmark migration actions only; ordinary running user jobs remain excluded.
Adjacent-domain scheduler bridge	Decima-style Spark-DAG simulator bridge, registered non-GPU scheduling diagnostics, and same-domain simulator notes.	Shows adjacent-domain coverage; it is not a GPU co-location full-stack comparison.
Learning and hidden-regime extensions	Active-bucket concentration, forced exploration, dwell/switching, detector-delay, optional live-hook records, and regime-detector fields.	Extension evidence; the current default live scheduler does not claim a complete online-learning theorem.
Non-future closure and future-state safety	Conservative all-state safety cover, probe/defer admission, zero theorem service for unknown states, and non-future closure rows excluding arbitrary future workloads.	Prevents overclaiming positive service or superiority for arbitrary future workloads, unregistered schedulers, or unmeasured hardware states.

This table keeps external scheduler and extension evidence visible while preserving the paper's scoped mathematical claim.

Code and Data Availability

The code, experiment artifacts, measured service-cache summaries, reviewer supplement manifest, and README files for this submission are provided in the supplementary Scheduleurm artifact package. The package contains the Scheduleurm source tree, theorem-facing replay modules, measured service-cache files, generated experiment summaries, and the Lean supplement headed by `ScheduleurmUpload.lean`. The reproduction script rebuilds the paper PDF, reruns the safe replay and certificate checks, executes the targeted tests, and refreshes the reproduction manifest. Live launches and external full-stack runtime probes are not executed by default; their recorded outputs are packaged with the corresponding environment notes. Raw scheduler history records that are outside the theorem-facing completed-active population are retained as operational telemetry and are not used as the claimed replication population. The Lean supplement package contains `ScheduleurmUpload.lean`, `lakefile.toml`, `lean-toolchain`, `build.log`, `theorem_crosswalk.md`, `manifest.json`, and checksums.

References

- Altman E, Avrachenkov K, Ayesta U (2006) A survey on discriminatory processor sharing. *Queueing Systems* 53(1–2):53–63, URL <http://dx.doi.org/10.1007/s11134-006-7586-8>.
- Bekker R, Borst SC (2006) Optimal admission control in queues with workload-dependent service rates. *Probability in the Engineering and Informational Sciences* 20(4):543–570, URL <http://dx.doi.org/10.1017/S0269964806060335>.
- Carvalho MNL, Simitsis A, Queralt A, Romero O (2024) Workload placement on heterogeneous CPU-GPU systems. *Proceedings of the VLDB Endowment* 17(12):4241–4244, URL <http://dx.doi.org/10.14778/3685800.3685845>.
- Chen W, Mo Z, Xu H, Ye K, Xu C (2023) Interference-aware multiplexing for deep learning in GPU clusters: A middleware approach. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (ACM), URL <http://dx.doi.org/10.1145/3581784.3607060>.
- Correcher JF, Perea F, Alvarez-Valdes R (2024) The berth allocation and quay crane assignment problem with crane travel and setup times. *Computers & Operations Research* 162:106468, URL <http://dx.doi.org/10.1016/j.cor.2023.106468>.
- Dai JG, Gluzman M (2022) Queueing network controls via deep reinforcement learning. *Stochastic Systems* 12(1):30–67, URL <http://dx.doi.org/10.1287/stsy.2021.0081>.
- Dai JG, Meyn SP (1995) Stability and convergence of moments for multiclass queueing networks via fluid limit models. *IEEE Transactions on Automatic Control* 40(11):1889–1904, URL <http://dx.doi.org/10.1109/9.471210>.

- de Moura L, Ullrich S (2021) The Lean 4 theorem prover and programming language. *Automated Deduction – CADE* 28, volume 12699 of *Lecture Notes in Computer Science*, 625–635 (Springer), URL http://dx.doi.org/10.1007/978-3-030-79876-5_37.
- Gupta V, Zhang J (2022) Approximations and optimal control for state-dependent limited processor sharing queues. *Stochastic Systems* 12(2):205–225, URL <http://dx.doi.org/10.1287/stsy.2021.0087>.
- Hurink J, Jurisch B, Thole M (1994) Tabu search for the job-shop scheduling problem with multi-purpose machines. *OR Spectrum* 15(4):205–215, URL <http://dx.doi.org/10.1007/BF01719451>.
- Kacem I, Hammadi S, Borne P (2002) Pareto-optimality approach for flexible job-shop scheduling problems: Hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and Computers in Simulation* 60(3–5):245–276, URL [http://dx.doi.org/10.1016/S0378-4754\(02\)00019-8](http://dx.doi.org/10.1016/S0378-4754(02)00019-8).
- Kolisch R, Sprecher A (1997) PSPLIB: A project scheduling problem library. *European Journal of Operational Research* 96(1):205–216, URL [http://dx.doi.org/10.1016/S0377-2217\(96\)00170-1](http://dx.doi.org/10.1016/S0377-2217(96)00170-1).
- Le TN, Sun X, Chowdhury M, Liu Z (2020) AlloX: Compute allocation in hybrid clusters. *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16 (ACM), URL <http://dx.doi.org/10.1145/3342195.3387547>.
- Liu B, Xie Q, Modiano E (2022) RL-QN: A reinforcement learning framework for optimal control of queueing systems. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 7(1):1–35, URL <http://dx.doi.org/10.1145/3529375>.
- Mahajan K, Balasubramanian A, Singhvi A, Venkataraman S, Akella A, Phanishayee A, Chawla S (2020) Themis: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 289–304 (USENIX Association), URL <https://www.usenix.org/conference/nsdi20/presentation/mahajan>.
- Mao H, Schwarzkopf M, Venkatakrishnan SB, Meng Z, Alizadeh M (2019) Learning scheduling algorithms for data processing clusters. *Proceedings of the ACM Special Interest Group on Data Communication*, 270–288 (ACM), URL <http://dx.doi.org/10.1145/3341302.3342080>.
- Meyn SP, Tweedie RL (2009) *Markov Chains and Stochastic Stability* (Cambridge University Press), 2 edition, URL <http://dx.doi.org/10.1017/CB09780511626630>.
- Narayanan D, Santhanam K, Kazhamiaka F, Phanishayee A, Zaharia M (2020) Heterogeneity-aware cluster scheduling policies for deep learning workloads. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*, 481–498 (USENIX Association), URL <https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak>.
- Neely MJ (2010) *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Synthesis Lectures on Communication Networks (Morgan & Claypool Publishers), URL <http://dx.doi.org/10.2200/S00271ED1V01Y201006CNT007>.

- Peng Y, Bao Y, Chen Y, Wu C, Meng C, Lin W (2019) DL2: A deep learning-driven scheduler for deep learning clusters. *arXiv preprint arXiv:1909.06040* URL <https://arxiv.org/abs/1909.06040>.
- Perron L, Didier F, Gay S (2023) The CP-SAT-LP solver. *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *Leibniz International Proceedings in Informatics*, 3:1–3:2 (Schloss Dagstuhl–Leibniz-Zentrum für Informatik), URL <http://dx.doi.org/10.4230/LIPIcs.CP.2023.3>.
- Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP (2021) Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 1–18 (USENIX Association), URL <https://www.usenix.org/conference/osdi21/presentation/qiao>.
- Scully Z, Harchol-Balter M, Scheller-Wolf A (2020) Simple near-optimal scheduling for the M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4(1):1–29, URL <http://dx.doi.org/10.1145/3379477>.
- Stolyar AL (2004) MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *The Annals of Applied Probability* 14(1):1–53, URL <http://dx.doi.org/10.1214/aoap/1075828046>.
- Subramanya SJ, Arfeen D, Lin S, Qiao A, Jia Z, Ganger GR (2023) Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 642–657 (ACM), URL <http://dx.doi.org/10.1145/3600006.3613175>.
- Tassiulas L, Ephremides A (1992) Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, URL <http://dx.doi.org/10.1109/9.182479>.
- Xiao W, Bhardwaj R, Ramjee R, Sivathanu M, Kwatra N, Han Z, Patel P, Peng X, Zhao H, Zhang Q, Yang F, Zhou L (2018) Gandiva: Introspective cluster scheduling for deep learning. *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, 595–610 (USENIX Association), URL <https://www.usenix.org/conference/osdi18/presentation/xiao>.
- Yang Z, Srikant R, Ying L (2023) Learning while scheduling in multi-server systems with unknown statistics: MaxWeight with discounted UCB. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4275–4312 (PMLR), URL <https://proceedings.mlr.press/v206/yang23d.html>.
- Yu P, Chowdhury M (2020) Salus: Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, volume 2, 600–615, URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/d9cd83bc91b8c36a0c7c0fcc59228f2-Abstract.html.